

RESEARCH ARTICLE

Singularity-Free Guiding Vector Field Over Bézier's Curves Applied to Rovers Path Planning and Path Following

Alfredo González-Calvin | Lía García-Pérez | Juan F. Jiménez

Dept. Arquitectura de Computadores y Automática, Facultad de Ciencias Físicas, Universidad Complutense, Madrid, Spain

Correspondence: Alfredo González-Calvin (alfredgo@ucm.es)

Received: 7 December 2023 | **Revised:** 30 December 2024 | **Accepted:** 16 February 2025

Funding: This work is partially supported by IA-GES-BLOOM-CM (Y2020/TCS-6420) of the Synergic Projects program of the Autonomous Community of Madrid and INSERTION (PID2021-27648OB-C33) of the Knowledge Generation program of the Ministry of Science and Innovation.

Keywords: curvature-changing speed setpoint | guiding vector fields | parametric paths | path following | rover | speed controller | wheeled mobile robots

ABSTRACT

This paper presents a guidance algorithm for solving the problem of following parametric paths, as well as a curvature-varying speed setpoint for land-based car-type wheeled mobile robots (WMRs). The guidance algorithm relies on singularity-free guiding vector fields (SF-GVFs). This novel guiding vector field (GVF) approach expands the desired robot path and the GVF to a higher dimensional space, in which an angular control function can be found to ensure global asymptotic convergence to the desired parametric path while avoiding field singularities. In SF-GVF, paths should follow a parametric definition. This feature makes using Bezier's curves attractive to define the robot's desired path. The curvature-varying speed setpoint, combined with the guidance algorithm, eases the convergence to the path when physical restrictions exist, such as minimal turning radius or maximal lateral acceleration. We provide theoretical results, simulations, and outdoor experiments using a WMR platform assembled with off-the-shelf components. The small rover (WMR) selected provides an easy-to-use nonholonomic platform for the experiments. The results could be extrapolated to full-scale or more complex vehicles, providing the necessary vehicle control system adaptations, while the GVF algorithm would remain the same.

1 | Introduction

Autonomous mobile robots (AMRs) have become a broad research area with applications in many fields. From automatic storage systems in warehouses, environment monitoring, postcatastrophe inspection, and emergency rescue to planetary exploration, the presence of AMRs is ubiquitous. A review of the categories and applications of AMRs can be found in Rubio et al. (2019).

Following a classical scheme (Siegwart et al. 2011, p. 10), the mobility control of the AMR could be divided into two main tasks: perception and motion control. We are interested in

motion control since it makes the AMR follow a planned path (s) (path following).

There are certain types of AMR applications in which the robots are required to pass through specified points that an expert user defines. For example, environmental monitoring robots (Hitz et al. 2017) that take samples or measurements at points of interest, surveillance robots that need to visit specific locations (Thakur et al. 2013), agricultural robots that must apply treatments at designated locations (Bechar and Vigneault 2016, 2017), or delivery robots (Mathew et al. 2015) that have to deliver products to defined locations. Usually, the

robot's human operators have prior knowledge of the mission. They can plan the robot's path by breaking it into feasible trajectories. This can be done using maps or photographs of the terrain the robots will traverse and defining a set of waypoints that the rover should visit or at least pass close to. The robot, in turn, must generate and follow a trajectory that ensures these points are reached. In some cases, it is helpful for the user to define, in addition to the waypoints, an area within which the trajectory is restricted, for example, for safety reasons. Think, for instance, about rovers for outdoor exploration or patrol. They should be able to cope with unstructured environments where slopes, terrain irregularities, and obstacles could hamper the rover's performance.

However, for a robot to be able to follow a path visiting specific waypoints autonomously, it is not enough to define the trajectory between them. To make it an autonomous system, an AMR must be able to drive itself, be aware of its state, avoid obstacles, and so forth, and ultimately reach its predetermined goals. For this purpose, once a suitable trajectory has been defined for the AMR, it is essential to provide a motion control algorithm that allows it to follow the trajectory correctly. The control algorithm can be divided into Guidance and Navigation.

The guidance controller provides the desired setpoints (e.g., position, heading, and speed) to take the robot from its current state (i.e., position, attitude, and speed) to the desired trajectory. When planning a trajectory, we may consider the mission the robot has to accomplish as the starting point. This planning process must take into account the kinematic and dynamic constraints. Our robot is a typical nonholonomic system, a (four) wheeled mobile robot (WMR), with a minimum turning radius and maximum acceleration limits to avoid wheel slippage, lateral skidding, or robot overturning. The latter two could be estimated using an Inertial Unit on board (Nourizadeh et al. 2023).

In some cases, the upper and lower speed limits are mission-dependent, while in others, the speed limit is fixed during the whole mission. Using the first approach, we can facilitate the operator's search for feasible trajectories by linking the speed control to the curvature of the trajectory, thus connecting it to the guidance controllers. Therefore, it is sufficient to define waypoints to design a reasonable path, avoiding significant obstacles and sharp turns, and leave it to the guidance system to adjust the speed to follow the resulting trajectory.

The AMR navigation comprises headings and speed control to allow the robot to follow the desired trajectory. It is linked to the mechanical characteristics of the robot and the specific onboard actuators and sensors.

This paper outlines the development, simulation, and field experimentation of a navigation system based on waypoints. This system is implemented and tested on a small, land-based, four-WMR with a low-cost onboard electronic system. We give this particular type of WMR the generic name of *rover* because it can move over rough terrain.

The rover has to pass through the waypoints determined by the user. These waypoints are connected by Bézier curves,

whose first and last points are the waypoints, while the rest of them (control points) allow the route to remain within safe navigation zones. The rover's task is to follow the trajectory calculated using the Bézier curves. The user can modify waypoints and control points during the mission and send them to the rover. When the rover receives them, a new trajectory is generated. The path-following system, consisting of guidance and speed control, computes the appropriate commands to enable the rover to follow the trajectory through the waypoints. We use guiding vector fields (GVFs), in particular, the novel singularity-free guiding vector field (SF-GVF) methodology (Yao 2021), to set the heading and the curvature of the trajectory to set the speed. A nonlinear control strategy for the heading ensures mathematical convergence to the path. In addition, we use a linear controller for the speed.

As we shall describe later in the paper, SF-GVF requires defining parametric curves to determine the robot's path. However, this can be challenging in real-world applications, especially when the robot moves in nonstructured environments. To address this issue, we propose using Bezier curves, which are parametric curves well-suited for defining the robot's path in SF-GVF. This approach can make it easier to apply SF-GVF to real-world scenarios. The user only needs to specify the control points for the Bezier curve and then check whether the resulting path meets their requirements.

Results of the guidance and navigation controllers are presented first in simulation and after with a real rover in a natural environment. In both cases, the paparazzi environment was used (Gati and Balazs 2013). Paparazzi autopilot has been used together with GVF for unmanned aerial vehicles (UAVs) (Kapitanyuk et al. 2017; De Marina et al. 2017).

In the following sections of the paper, we will progressively develop our proposal according to the following sequence: Section 2 describes a simple rover model that will be employed later to build the guidance control system. Section 3 deals with the SF-GVF guidance approach, the rover steering control description, and its stability properties. Section 4 explains the use of Bézier curves in path planning and discusses their main advantages and disadvantages. Section 5 presents the speed controller used in our rover and the curvature-varying setpoint. Section 6 is devoted to simulated and experimental results, briefly introducing the paparazzi ecosystem, an open-source software platform we use to develop rover onboard code and simulate and monitor experiments with a team of small rovers. Eventually, we close the work in Section 7, drawing some conclusions.

2 | Rover Model

Our rover is a four-WMR with a spring suspension system to ensure maximum contact between the wheels and the ground. Two fixed rear wheels on a joint axle and two front steering wheels, a car-like or type (1, 1) according to (Siciliano and Khatib 2008) robot classification.

2.1 | Basic Model Assumptions

Figure 1 depicts an outline of a four-wheeled vehicle. As it is known, a four-wheeled car will not skid if the longitudinal axis of each wheel is aligned with the circumferences they trace around to the center of rotation. For this purpose, an Ackermann steering linkage can be employed, in which each wheel turns a different angle when tracing a curve, according to well-known equations (Siciliano and Khatib 2008). However, when conditions are ideal (i.e., no skid), the two wheels of an Ackerman drive can be replaced by a virtual single wheel situated at the center of the front axis, that is, in the direction of motion of the rover. Therefore, in Figure 1, the angle θ represents the rover's heading in an inertial frame $W(p)$, $p = (p_x, p_y) \in \mathbb{R}^2$ attached to the earth and the angle ϕ is the angle between the longitudinal axis of the car (i.e., its direction of motion) and the virtual single wheel (shown in red). Then, the velocity can be computed as

$$\begin{cases} \dot{p}_x = v \cos(\theta), \\ \dot{p}_y = v \sin(\theta), \end{cases} \quad (1)$$

where v is the speed in the direction of motion. Now let r be the distance traveled by the car (by its rear axis), and let R be the instantaneous radius of rotation. An infinitesimal displacement in the distance implies $dr = R d\theta$, and it can be seen that $R = \frac{l}{\tan(\phi)}$. Then $\dot{r} = v = \frac{l}{\tan(\phi)} \dot{\theta}$, where l represents the wheelbase of the vehicle. Finally, assume that the control input is the angular rotation of the virtual front wheel $\dot{\phi} = u_\phi$, then the model can be represented as

$$\begin{cases} \dot{p}_x = v \cos(\theta), \\ \dot{p}_y = v \sin(\theta), \\ \dot{\theta} = v \frac{\tan \phi}{l}, \\ \dot{\phi} = u_\phi \end{cases} \quad (2)$$

with the condition that $-\frac{\pi}{2} < \phi < \frac{\pi}{2}$ to stay in the approximation of no skid. We may consider $-\phi_{\max} < \phi < \phi_{\max}$, with $\phi_{\max}$, the maximum turn angle the wheels can rotate around their vertical axis. Usually, $\phi_{\max} \approx \frac{\pi}{6}$.

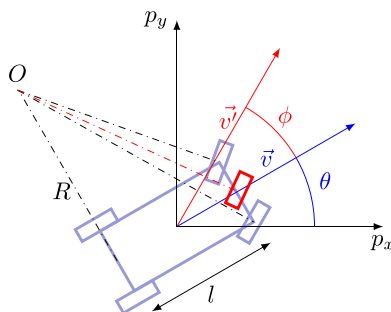


FIGURE 1 | Representation of a simple four-wheeled car with wheelbase l . The car is shown in light blue, while the virtual front wheel is shown in red. The direction of motion is shown with a blue line, and the direction of the virtual wheel is shown in red.

2.2 | Rover Kinematic Equations

If the control signal u_ϕ can modify the value of ϕ practically instantaneously (i.e., $\phi = u_\phi$), then we can consider for the rover the following nonholonomic model:

$$\begin{cases} \dot{p}_x = v \cos(\theta), \\ \dot{p}_y = v \sin(\theta), \\ \dot{\theta} = u_\theta \end{cases} \quad (3)$$

and the angle of the virtual front wheel can be computed as $\phi = \arctan\left(\frac{l u_\theta}{v}\right)$.

Eventually, we arrive at a simple unicycle model. Besides, we conducted experiments using small RC rovers propelled by electrical motors, so we can take $v = u_v$, acting directly on the rover's speed, disregarding inertial, and resistance to advance.

3 | Parametric GVF for Rover Guidance and Path Following

Many algorithms for path following can be found in the literature. Among them, some of the most common are carrot chasing (Safwat et al. 2018), line-of-sight (Gu et al. 2022), or pure pursuit (Samuel et al. 2016). A complete survey and comparison can be found in Sujit et al. (2014), where authors show that using simulations, vector field (VF) is the technique that more accurately follows the path. We use the GVFs methodology to guide the rovers. At each point in space, the GVF provides a suitable heading direction for the rover to converge towards the target path or remain on it when reached. This work is based on a novel methodology, singularity-free parametric GVF (SF-GVF) (Yao et al. 2021), which extends GVF to deal with self-intersecting paths and singularities (i.e., points at which the VF is null) by defining parametric curves in an augmented space, where all singular points are removed.

Parametric GVF was designed to work with paths described by generic parametric equations in $\mathbb{R}^3$ and $\mathbb{R}^2$. In this last case,

$$\mathcal{P} := \{p_x = f_1(w), p_y = f_2(w)\}, \quad (4)$$

where w is the parameter of the path, $p_x, p_y \in \mathbb{R}^2$ and $f_1, f_2 \in C^2$. Therefore, even though we are working with Bézier curves, any parametric curve could be used.

3.1 | Guidance Problem

The GVF should be designed so that their integral curves converge to the desired path. Thus, if the vehicle aligns with the VF, it will converge to the desired path. This approach is known as the vector-field guided path-following navigation problem; it can be briefly described as follows (Yao et al. 2021):

Problem 1 (The vector-field guided path-following navigation problem). *Given a desired path $\mathcal{P} \subset \mathbb{R}^n$, to design*

a continuous differentiable vector field $\chi : \mathbb{R}^n \rightarrow \mathbb{R}^n$ such that the dynamical system $\dot{\xi}(t) = \chi(\xi(t))$ fulfills the following two conditions:

1. There is a neighborhood of $\mathcal{D} \subseteq \mathbb{R}^n$ such that for all $\xi(0) \in \mathcal{D}$, the distance between $\xi(t)$ and $\mathcal{P}$ approaches zero as $t \rightarrow \infty$.
2. If a trajectory starts on the desired path, it remains in the desired path. $\xi(0) \in \mathcal{P} \Rightarrow \xi(t) \in \mathcal{P}, \forall t \geq 0$.

To design an appropriate SF-GVF, we first define the following surfaces, departing from the definition of the desired path $\mathcal{P}$ in (4),

$$\phi_1(\xi) = p_x - f_1(w), \quad \phi_2(\xi) = p_y - f_2(w), \quad (5)$$

where $\xi = (p_x, p_y, w) \in \mathbb{R}^3$ represents a vector on an “augmented” space (i.e., the augmented space consists of the rover position (p_x, p_y) and the curve parameter w). We then define an “augmented path” as

$$\text{aug}\mathcal{P} := \{\xi = (p_x, p_y, w) \in \mathbb{R}^3 : \phi_i = 0, i = 1, 2\}. \quad (6)$$

Figure 2 shows an example of a path $\mathcal{P}$ defined using a Bézier curve. Notice that $\text{aug}\mathcal{P}$ is the intersection of the surfaces $\phi_1(\xi)$ and $\phi_2(\xi)$ and $\mathcal{P}$ is just the projection of $\text{aug}\mathcal{P}$ onto (p_x, p_y) . The figure shows one of the main features of SF-GVF. The actual desired path $\mathcal{P}$ for the rover (blue line) has a crossing point, and it is impossible to define a single vector for the rover to follow to stay on the path at such a point. The augmented path $\text{aug}\mathcal{P}$ instead unfolds $\mathcal{P}$ using the parameter w . Thus, there is a single three-dimensional (3D) vector for every point of the 3D

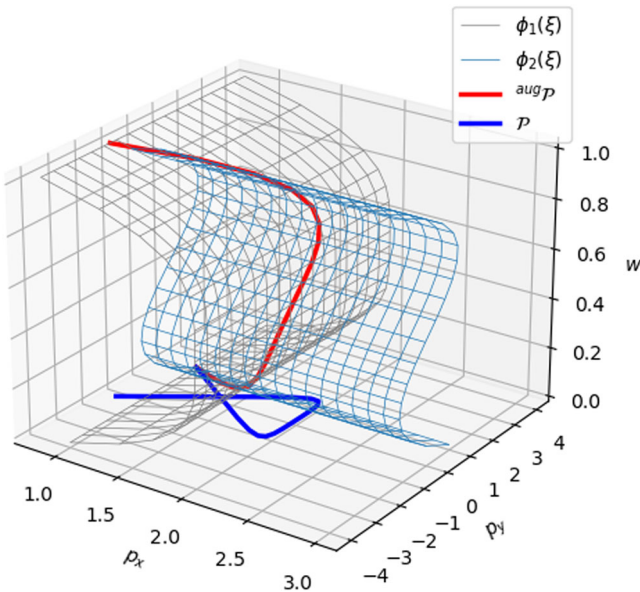


FIGURE 2 | Illustration of the relationship among $\mathcal{P}$, $\text{aug}\mathcal{P}$, $\phi_1(\xi)$, and $\phi_2(\xi)$. $\mathcal{P}$ is a Bézier's curve of degree 4, with control points $\mathcal{C} = \{(1, 1), (3, -4), (1, 0), (3, 4), (1, -1)\}$.

unfolded curve. The rover can now be guided using its two-dimensional (2D) projection onto the plane p_x, p_y .

We consider the function $e(\xi) = (\phi_1(\xi), \phi_2(\xi))^T : \mathbb{R}^3 \rightarrow \mathbb{R}^2$ an error function, which gives a measurement of the splitting between a point ξ and the path $\mathcal{P}$, $\|e(\xi)\| = 0 \Rightarrow \xi \in \mathcal{P}$ (where $\|\cdot\|$ is the Euclidean norm). We can now obtain a GVF on the “augmented” space, departing from $\phi_1(\xi)$ and $\phi_2(\xi)$,

$$\text{aug}\chi = \nabla\phi_1 \times \nabla\phi_2 - \sum_{i=1}^2 k_i \phi_i \nabla\phi_i. \quad (7)$$

The first addend of Equation (7) is perpendicular to $\nabla\phi_1$ and $\nabla\phi_2$. So, it is a vector tangential to the surfaces $\phi_1(\xi) = 0$ and $\phi_2(\xi) = 0$, pointing along $\text{aug}\mathcal{P}$. It is called in Yao et al. (2021) the *propagation term*. The sense of this propagation term can be changed just by swapping the gradients in the cross-product. The second addend points towards that surface (i.e., it is the normal component of the VF) and, therefore, towards $\text{aug}\mathcal{P}$. This term is called the *converging term*, where $k_i > 0$ are adjustable gains to fit the normal component of the field.

The gradients $\nabla\phi_1 = (1, 0, -f'_1(w))^T$ and $\nabla\phi_2 = (0, 1, -f'_2(w))^T$ can be straightforwardly obtained, where $f'_i(w) := \frac{df_i(w)}{dw}$. Besides,

$$\nabla\phi_1 \times \nabla\phi_2 = (f'_1(w), f'_2(w), 1). \quad (8)$$

Thus, our “augmented vector field” is

$$\text{aug}\chi(\xi) = \begin{pmatrix} f'_1(w) - k_1\phi_1 \\ f'_2(w) - k_2\phi_2 \\ 1 + k_1\phi_1 f'_1(w) + k_2\phi_2 f'_2(w) \end{pmatrix}. \quad (9)$$

Note that the third component of the vector in Equation (8) is always constant and equal to 1, regardless of the path's parametrization. Moreover, the terms $\nabla\phi_1 \times \nabla\phi_2$ and $\sum_{i=1}^2 k_i \phi_i \nabla\phi_i$ in Equation (7) are orthogonal to each other and are linearly independent. As a result, the third component of $\text{aug}\chi(\xi)$ is always nonzero. Therefore, $\text{aug}\chi(\xi) \neq 0$ for all $\xi \in \mathbb{R}^3$. Thus, there are no singular points in the augmented field.

In summary, we have added a fictitious third coordinate to the dimensions of our problem, including the path parameter w as a new variable. This has several advantages (Yao et al. 2021). First, it constructs a GVF that leads the vehicle towards the path. Besides, the augmented path is not self-intersecting, although the path could be, see Figure 2, and the augmented GVF does not have singular points.

The vector field $\text{aug}\chi$ defined in Equation (7) is an effective solution to Problem 1, that is, $\xi(t) = \text{aug}\chi$ approaches the augmented path $\text{aug}\mathcal{P}$ if $\xi(0) \notin \text{aug}\mathcal{P}$ or remains in $\text{aug}\mathcal{P}$ if $\xi(0) \in \text{aug}\mathcal{P}$. Formal proofs can be found in Yao et al. (2018, Theorem 3) and Yao and Cao (2020, Proposition 2 and Theorem 2). Finally, Yao et al. (2021, Theorem 2) proved that if $\mathcal{P}$ is parameterized as in Equation (4), ϕ_1 and ϕ_2 are chosen like in

(5) and there are no singular points in ${}^{\text{aug}}\chi$, the projected trajectory $\xi^p = (p_x, p_y)$ globally converges to the path $\mathcal{P}$.

We will offer here a proof sketch of the exponential convergence of ${}^{\text{aug}}\chi(\xi)$ to ${}^{\text{aug}}\mathcal{P}$, according to the following proposition:

Proposition 1. *Let $\xi(t)$ be the solution to $\dot{\xi}(t) = {}^{\text{aug}}\chi(\xi(t))$ with ${}^{\text{aug}}\chi(\xi(t))$ as defined in Equation (9), then $\xi(t)$ will converge to the augmented path ${}^{\text{aug}}\mathcal{P}$, defined in Equation (6) as $t \rightarrow \infty$.*

Proof. The distance from the solution to the path at any time is defined as

$$\text{dist}(\xi(t), \mathcal{P}) = \inf\{\|\xi(t) - p\| : p \in {}^{\text{aug}}\mathcal{P}\}. \quad (10)$$

This distance can be approximated by $\|e(\xi(t))\|$ because the norm of $e(\xi) = (\phi_1(\xi), \phi_2(\xi))^T$ is bounded and there are not singular points in ${}^{\text{aug}}\mathcal{P}$. Taking the $e(\xi)$ time derivative

$$\begin{aligned} \dot{e}(\xi(t)) &= N^T(\xi(t)) \cdot \dot{\xi}(t) = N^T(\xi(t)) \cdot \chi(\xi(t)) \\ &= -N^T(\xi(t)) \cdot N(\xi(t)) \cdot K \cdot e(\xi(t)), \end{aligned} \quad (11)$$

where $K = \text{diag}(k_1, k_2)$, $N = (\nabla\phi_1, \nabla\phi_2)$, and $N^T \cdot (\nabla\phi_1 \times \nabla\phi_2) = 0$.

Now we define the following Lyapunov's function candidate:

$$V(\xi(t)) = \frac{1}{2}e^T(\xi(t)) \cdot K \cdot e(\xi(t)). \quad (12)$$

One important property of $V(\xi(t))$ and the error vector norm $\|e(\xi(t))\|$ is

$$2\frac{V}{k_m} \geq \|e\|^2 \geq 2\frac{V}{k_M}, \quad (13)$$

where $k_m = \min\{k_1, k_2\}$ and $k_M = \max\{k_1, k_2\}$. We will omit $\xi(t)$ dependence for clarity. Taking the time derivative of the Lyapunov function,

$$\begin{aligned} \dot{V} &= \frac{1}{2}(\dot{e}^T K e + e^T K \dot{e}) = -\frac{1}{2}(e^T K N^T N K e + e^T K N^T N K e) \\ &= -e^T Q e = -\|N K e\|^2. \end{aligned} \quad (14)$$

Notice that $Q = K N^T N K > 0$, $\det(Q) = k_1^2 k_2^2 \|\nabla\phi_1 \times \nabla\phi_2\|^2 \geq 0$ (recall Equation 9). Thus $\dot{V} = 0 \Rightarrow e(\xi) = 0$. Let $\lambda_{\min} > 0$ be the minimum eigenvalue of Q . Then,

$$\dot{V} \leq -\lambda_{\min} \|e\|^2 \leq -2\lambda_{\min} \frac{V}{k_M}. \quad (15)$$

Thus,

$$\frac{k_m}{2} \|e\|^2 \leq V \leq V(e_0) \exp\left(-\frac{2\lambda_{\min} t}{k_M}\right) \leq \frac{k_M}{2} \|e_0\|^2. \quad (16)$$

With $e_0 = e(\xi(0))$, the initial error. Eventually,

$$\|e\| \leq \sqrt{\frac{k_M}{k_m}} \|e_0\| \exp\left(-\frac{\lambda_{\min} t}{k_M}\right). \quad (17)$$

□

3.2 | Path Following

Once a dynamic model for the rover, a desired path, and its GVF associated have been defined, we need to find a control law that makes the rover trajectory converge to the desired path. First, we extend the dynamic model described in Section 2, Equation (3), adding an equation for the new variable w :

$$\dot{w} = v \frac{\chi_3}{\sqrt{\chi_1^2 + \chi_2^2}}, \quad (18)$$

where ${}^{\text{aug}}\chi = (\chi_1, \chi_2, \chi_3)^T$ are the components of the “augmented” guiding field. The path parameter must adapt accordingly to the rover's speed because the rover will follow the GVF evaluated at a specific w . As discussed in Yao (2021, p. 194), the GVF path following process could also be considered as a special trajectory tracking algorithm (Siciliano et al. 2010, p. 506), in which the tracking parameter is state-dependent, closing a control loop.

Therefore, the objective should be to align the rover augmented velocity $\dot{\xi} = (\dot{p}_x, \dot{p}_y, \dot{w})^T$ with the GVF, making it head in the direction of the field. Assuming the disturbances to the system can be neglected, we approximate the rover velocity direction with its heading vector, $\dot{p} \approx \dot{h} = (\cos(\theta), \sin(\theta))^T$, where $\hat{\cdot}$ represents the normalization operator, and define χ^p as the projection of the guiding field ${}^{\text{aug}}\chi$ onto the plane (p_x, p_y) ,

$$\chi^p := (\hat{\chi}_1, \hat{\chi}_2)^T = \frac{1}{\sqrt{\chi_1^2 + \chi_2^2}} \begin{pmatrix} \chi_1 \\ \chi_2 \end{pmatrix}. \quad (19)$$

Figure 3 shows the concepts relating to the augmented and projected VF and trajectories. It is interesting to notice how the projected VF changes following the parameter w .

Hence, let us consider the following error function between the rover's orientation and $\hat{\chi}^p$:

$$e_* = \begin{pmatrix} \hat{h} - \hat{\chi}^p \\ \dot{w} - v \frac{\chi_3}{\|\chi^p\|} \end{pmatrix} = \begin{pmatrix} \hat{h} - \hat{\chi}^p \\ 0 \end{pmatrix}, \quad (20)$$

which is $\mathbf{0}$ if and only if $\hat{h}^T \hat{\chi}^p = 1$; therefore, the orientation is aligned with the VF. It is clear that thanks to the election made of $\dot{w}$, we only have to find a control law that makes $\hat{h} - \hat{\chi}^p \rightarrow 0$, that is, a control law that asymptotically aligns the rover with the projection of the augmented guiding field, χ^p . The variation of χ^p orientation as the rover moves can be obtained as

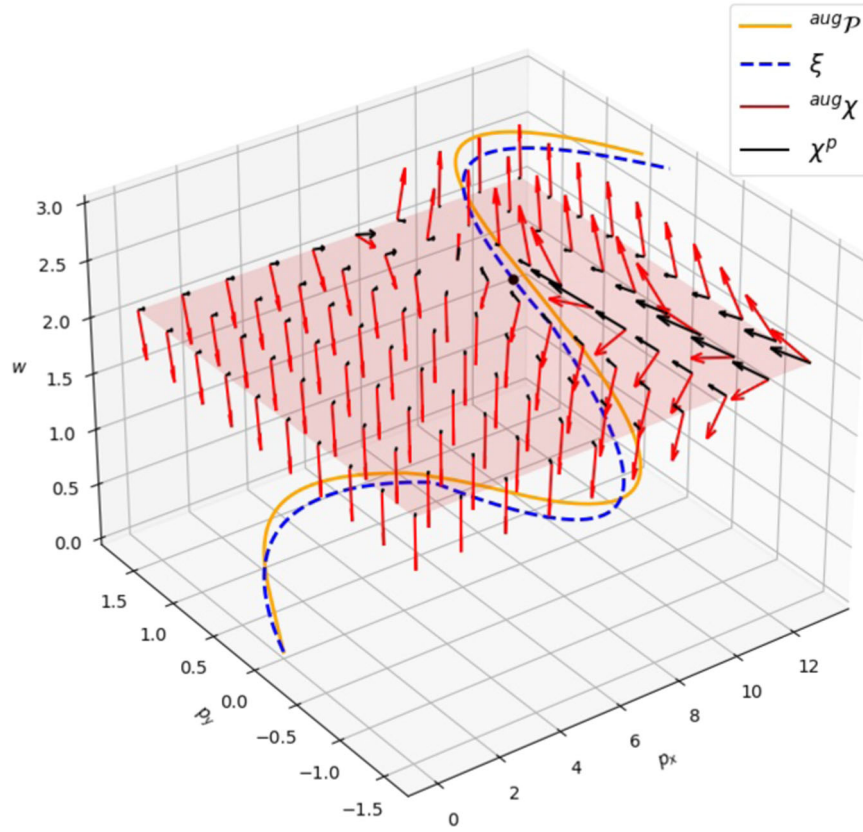


FIGURE 3 | Illustration of the relationship between $^{\text{aug}}\chi$ and χ^p . The vector field χ^p (shown as black arrows) is the projection of $^{\text{aug}}\chi$ (shown as red arrows) onto the plane (p_x, p_y) of constant w (shown as a black dot in the red plane). The solid orange line represents the augmented path, while the dashed blue line represents the augmented trajectory ξ .

$$\frac{d}{dt}\hat{\chi}^p = \frac{1}{\|\hat{\chi}^p\|} (I - \hat{\chi}^p \hat{\chi}^{pT}) \frac{d\chi^p}{dt} = \frac{1}{\|\hat{\chi}^p\|} (I - \hat{\chi}^p \hat{\chi}^{pT}) J(\chi^p) v,$$

where $I \in \mathbb{R}^2$ is the identity matrix and $J(\chi^p)$ is the Jacobian matrix of the field $\chi^p = (\chi_1, \chi_2)$ with respect to $p = (p_x, p_y)$. Notice also that $(I - \hat{\chi}^p \hat{\chi}^{pT}) = E \hat{\chi}^p (E \hat{\chi}^p)^T$ is a projector in the direction orthogonal to $\hat{\chi}^p$. So,

$$\frac{d}{dt}\hat{\chi}^p = - \left(\frac{\hat{\chi}^{pT}}{\|\hat{\chi}^p\|} E J(\chi^p) v \right) E \hat{\chi}^p = \dot{\theta}_d E \hat{\chi}^p,$$

where E is the 90° counterclock rotation matrix $E = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ and we define

$$\dot{\theta}_d := - \left(\frac{\hat{\chi}^{pT}}{\|\hat{\chi}^p\|} E J(\chi^p) v \right). \quad (21)$$

Notice that $\dot{\theta}_d$ is a scalar quantity, and it can be considered as the rotation rate of $\hat{\chi}^p$, since $E \hat{\chi}^p \perp \hat{\chi}^p$. Then, if the angular velocity of the rover $\dot{\theta}$ converges to $\dot{\theta}_d$ while $\hat{h} \rightarrow \hat{\chi}^p$, the rover will follow the GVF and, eventually, will connect to the desired path. A suitable Lyapunov function can be found in Yao et al. (2021) as $V = \frac{1}{2} \|e_*\|^2$, whose time derivative is

$$\begin{aligned} \dot{V} &= (\dot{\hat{h}} - \dot{\hat{\chi}}^p)^T (h - \hat{\chi}^p), \\ \dot{V} &= (-\dot{\theta} \hat{h}^T E + \dot{\theta}_d \hat{\chi}^{pT} E) (h - \hat{\chi}^p), \\ \dot{V} &= (\dot{\theta} - \dot{\theta}_d) (\hat{h}^T E \hat{\chi}^p), \end{aligned}$$

where $\dot{\hat{h}} = \dot{\theta} E \hat{h}$. Therefore, by choosing the control function

$$\dot{\theta} = u_\theta = \dot{\theta}_d - k_\theta \hat{h}^T E \hat{\chi}^p : k_\theta \in \mathbb{R}^+, \quad (22)$$

then $\dot{V} = -k_\theta (\hat{h}^T E \hat{\chi}^p)^2 \leq 0$. This means that $\hat{h} \rightarrow \hat{\chi}^p$ and $\dot{\theta} \rightarrow \dot{\theta}_d$ as desired. Notice that k_θ is an adjustable gain to modulate the control action.

We use spline segments with C^4 continuity, while the endpoints connecting two curves have C^2 continuity. This guarantees convergence, justifying using at least fifth-order Bézier curves.

3.3 | A Note on Field Perturbation

We now add a perturbation to the VF,

$$\dot{\xi} = \chi(\xi(t)) + d(t), \quad (23)$$

where $d : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^2$ is a bounded function of time $t, \forall t \geq 0$. Then, the dynamics of the path-following error, Equation (11), changes

$$\dot{e}(t) = N^T(\xi(t))(\chi(\xi(t)) + d(t)). \quad (24)$$

Using again Equation (12) as Lyapunov's candidate function, its time derivative is

$$\begin{aligned} \dot{V} &= \frac{1}{2}(\dot{e}^T K e + e^T K \dot{e}) = -\frac{1}{2}((\chi + d)^T N K e \\ &\quad + e^T K N^T(\chi + d)) = -\|N K e\|^2 + d^T N K e \end{aligned} \quad (25)$$

and using Young's inequality, $d^T N K e \leq \|N K e\|^2/2 + \|d\|^2/2$.

$$\dot{V} \leq -\frac{1}{2}\|N K e\|^2 + \frac{1}{2}\|d\|^2. \quad (26)$$

Following an identical reasoning as in Proposition 1 proof, we arrive to

$$\dot{V} \leq -\frac{1}{2}\lambda_{\min}\|e\|^2 + \frac{1}{2}\|d\|^2. \quad (27)$$

If $d(t)$ is bounded, $\|e\|$ will be bounded by $\sup_{t \in [0, \infty)} \|d(t)\|/\sqrt{\lambda_{\min}}$, while if $d(t)$ vanishes as $t \rightarrow \infty$, then $\|e(t)\| \rightarrow 0$ as $t \rightarrow \infty$.

4 | Bézier's Curves for Path Planning

Whatever mission a final user wants to assign to an AMR involves the explicit or implicit definition of trajectories the vehicle should cover to fulfill its objectives. Probably the broadest extended method is the generation of Dubins' splines based on Dubins' curves (Dubins 1957; Lavalle 2006, p. 880). For constant speed and predefined waypoints, a Dubins' spline is built by combining circle arcs and straight lines that minimize the time to cover the path. The method used a circle arc with the minimum radius allowed by the vehicle's speed. Another common approach is the use of N -splines. These are defined as 3rd third-degree polynomials, which interpolate two consecutive waypoints of the desired path. The polynomial coefficients become entirely determined by imposing continuity for the splines and their first derivatives on the common waypoints.

Even if SF-GVFs ensure convergence to any C^2 continuous parametric curve, not all parametric curves are suitable for an AMR mission. While the previously mentioned curves are quite common in literature, they lack several desired properties that ease the path creation and/or that allow one to fit the parametric curve to a real restricted area. That is, they usually lack: (1) an intuitive adjustment of the waypoints by the operator, (2) analytical knowledge of the area in which the curve is contained by a simple look at its waypoints, (3) ease of computation, and (4) ease of implementation in real hardware. The first property allows for fast path creation, while the second for an a priori known area in which the curve will be contained. The last two properties are related to hardware implementations, allowing the use of embedded memory and computationally restricted systems. Bézier's curves meet all these requirements, since (1) they are straightforward to adjust; (2) once we know

the waypoints of the Bézier curve, it is contained within its convex hull, so by joining the points of the Bézier curve with straight lines, we have the area in which the Bézier curve is contained; (3) they are easy to compute since they are well known and defined polynomials called Bernstein polynomials; and (4) for its implementation only its waypoints are needed, and no previous computations must be carried since its waypoints completely determine Bézier curves, as they are the coefficients of the polynomials. Thus, they are very suitable for AMR path following, as they can be easily defined and fit in tight areas, and are computationally friendly for embedded systems.

Bézier curves have occasionally been used in robotics (Han et al. 2010; Hilario et al. 2011; Jolly et al. 2009; Kawabata et al. 2015), but no examples of their use have been found in the literature for the design of safe trajectories in uncrewed vehicles except for berthing (Yuan et al. 2023). Bézier curves allow the creation of a trajectory simply and intuitively (Simba et al. 2016; Hwang et al. 2003). In addition, they are quite suitable for working in combination with GVFs, as we will see later on. A brief description of them is given to expose their main characteristics.

We focus on 2D Bézier curves, $f(w) \in \mathbb{R}^2$, of degree n . They are polynomials defined by a set of points, whose basis are the Bernstein polynomials $b_{k,n}(w) = \binom{n}{k} w^k (1-w)^{n-k} : w \in [0, 1]$, where $\binom{n}{k}$ is the binomial coefficient. A degree n Bézier curve is defined as

$$f(w) = (f_1(w), f_2(w)) = \sum_{k=0}^n \beta_k b_{k,n}(w), \quad (28)$$

where $\beta_k \in \mathbb{R}^2 : k \in [0, n]$ are the $n+1$ points that define the Bézier curve. Thus, as shown in Equation (28), there is no need to compute polynomial coefficients from the control points as, for instance, in standard basis cubic polynomials. This allows for less computations in the AMR vehicle, liberating resources for other tasks. Moreover, the points completely determine the shape of the curve, making it easier to adjust it manually, as in Han et al. (2010).

The curve begins at point β_0 and ends at point β_n , called *end points*, while $\beta_k : 1 \leq k \leq n-1$, called *control points*, are used to define the shape of the trajectory. For instance, two control points can shape the trajectory if a degree $n=3$ Bézier curve is used. It is important to note that the Bézier curve is contained within the convex hull created by its points. This is useful when restricting the curve to a specific area. Hence, Bézier curves allow for an easy and intuitive way to fit the curve into a desired or particular shape or area by (a) fixing the starting and end points and (b) moving the control points to create a safe trajectory, knowing that it will be contained in its convex hull. Finally, since Bézier curves are polynomials, an n -degree curve implies C^{n-1} continuity, meaning that when $n \geq 3$, the acceleration, velocity, and position are continuous. This property is necessary if curvature continuity is desired since it depends on the first and second derivatives of the parametric curve. Bézier curves have many other properties that make them useful for various applications; for a complete description, we refer to Hansford (2002).

Nevertheless, when generating paths, a single Bézier curve with many control points may be cumbersome for creating a desired shaped trajectory. Thus, a Bézier spline is designed for that purpose. When connecting two Bézier curves, the minimum requirement is to have C^0 continuity in the ending point β_n^i of the curve i and the starting point β_0^{i+1} of the curve $i + 1$ (except for the ending point of the last curve), that is, $\beta_n^i = \beta_0^{i+1}$ for every $i \in \{0, 1, \dots, N - 2\}$, where N is the number of Bézier curves that compose the spline. Moreover, if C^1 and C^2 continuities are desired, the following relationship between control points must hold:

$$\begin{cases} f^{i'}(1) = f^{i+1'}(0) \Rightarrow \beta_n^i - \beta_{n-1}^i = \beta_1^{i+1} - \beta_0^{i+1} \Rightarrow \\ \beta_1^{i+1} \stackrel{C^1 \text{ cont.}}{=} 2\beta_n^i - \beta_{n-1}^i \\ f^{i''}(1) = f^{i+1''}(0) \Rightarrow \beta_n^i - 2\beta_{n-1}^i + \beta_{n-2}^i \\ = \beta_2^{i+1} - 2\beta_1^{i+1} + \beta_0^{i+1} \Rightarrow \\ \beta_2^{i+1} \stackrel{C^1, C^2 \text{ cont.}}{=} 4\beta_n^i - 4\beta_{n-1}^i + \beta_{n-2}^i, \end{cases}$$

where $f^{i'} = \frac{df^i}{dw}$, $f^i(\cdot) = (f_1^i(\cdot), f_2^i(\cdot)) = \sum_{k=0}^n \beta_k^i b_{k,n}(\cdot)$ is the i th segment of the spline. This means that the first and second control points of the $i + 1$ curve ($\beta_1^{i+1}, \beta_2^{i+1}$) are also fixed and cannot be used to shape the trajectory. It can be shown that if C^p continuity is desired for a Bézier spline of degree $n > p$, it implies that p control points will be fixed, and only $n - 1 - p$ control points can be used to shape each segment of the spline. In our application, C^2 is necessary, and two control points would be enough. Therefore, $n = 5$ degree Bézier curves are selected.

A spline with N segments and C^2 continuity can be expressed as

$$f_s(w) = \begin{cases} f^0(w), & 0 \leq w \leq 1, \\ f^1(w - 1), & 1 \leq w \leq 2, \\ \vdots & \vdots \\ f^{N-1}(w - (N - 1)), & N - 1 \leq w \leq N, \end{cases}$$

where the continuity relations above hold for each Bézier curve. Note that $(w - i) \in [0, 1]$, as required in the definition of Bézier curves.

Thus, since we will work with N Bézier curves of degree $n = 5$, each curve (i.e., each segment of the spline) has C^4 continuity and, at the points in which two curves connect there is C^2 continuity. The following reasons justify the usage of this type of spline. First, the guidance algorithm requires that the complete trajectory is a path with at least C^2 continuity. Second, if lower degree curves were used (i.e., $n = 3$ and 4) and C^2 continuity is required, this would imply that less than two control points could be used to shape each segment of the spline, losing the ease of creating Bézier curves. Finally, only one segment and a higher degree Bézier curve could be used for the complete trajectory. However, in contrast to fixing ending points and only changing the i and $i + 1$ segment when moving a control point, creating a single polynomial leads not only to a complex adjustment of the curve to the desired area but to a higher-order polynomial computation, which could lead to lack of numerical stability (Faraway et al. 2007).

5 | Speed Control Problem

Several works can be found in the literature on adapting guidance controllers to the curvature, like in Leng and Minor (2017), for guidance algorithms to achieve better convergence. A speed controller for an AMR vehicle is helpful when following curvature-changing paths. This allows us to reach a constant speed setpoint and adapt the speed to path characteristics. The nonvarying setpoint provides for constant speed path-following while changing the setpoint depending on some information about the desired path (e.g., curvature) should speed up the convergence of the guidance controller. This approach is simple because only the desired reference speed changes; thus, whatever speed controller can be used, for example, in Cao et al. (2019), a finite time double sliding surface guidance algorithm is used for Subway's speed curve tracking.

In recent years, there has been increased attention towards determining trajectory-dependent speed profiles for guidance due to the rise of autonomous road vehicles (Gámez Serna and Ruichek 2017). Most works use a priori information about the desired trajectory to compute the path curvature and use this information to adapt vehicle speed, preventing slipping and rollover. Trajectory information can be extracted from maps, like, Villagra et al. (2012) that used a closed-form speed profiler combined with the path planner, providing a continuous velocity reference. Or (Li et al. 2012) that obtains the curvature using the Geographic Information Systems roadmaps. In other cases, certain assumptions are made about the curvature of roads to adapt the vehicle's speed, as in Park et al. (2015).

5.1 | Speed Controller

To control the rover speed, we use as a control signal u_v (i.e., the throttle) the output of a feedforward + PID controller:

$$u_v = k_f v_{\text{ref}} + k_p e_v(t) + k_d \dot{e}_v(t) + k_i \int_0^t e_v(\tau) d\tau, \quad (29)$$

where $v_{\text{ref}} \in \mathbb{R}_{\geq 0}$ is the desired speed, $e_v(t) = v_{\text{ref}} - v(t)$ is the error between this one and the actual speed of the rover, and $k_f, k_p, k_d, k_i \in \mathbb{R}_{\geq 0}$ are, respectively, the feedforward, proportional, derivative, and integral control constants. The feedforward controller allows reaching a constant speed depending on v_{ref} , while the proportional, integral, and derivative (PID) controller acts in the presence of error, allowing the desired speed to be reached without steady-state error. Recall that, as was pointed out in Section 2, we have ignored the dynamic of the rover. Thus $v \sim \propto u_v$ and we can consider

$$v = k_f v_{\text{ref}} + k_p (v_{\text{ref}} - v) + k_d (\dot{v}_{\text{ref}} - \dot{v}) + k_i \int_0^t (v_{\text{ref}} - v) dt. \quad (30)$$

Taking time derivatives, considering a constant v_{ref} and regrouping terms, we get a linear system:

$$\begin{pmatrix} \ddot{v} \\ \dot{v} \end{pmatrix} = \begin{pmatrix} -\frac{1+k_p}{k_d} & -\frac{k_i}{k_d} \\ 1 & 0 \end{pmatrix} \begin{pmatrix} \dot{v} \\ v \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \end{pmatrix} \frac{k_i}{k_d} v_{\text{ref}}. \quad (31)$$

It is immediate to see that the system matrix always has negative spectral abscissa, that is, the system is stable and $v \rightarrow v_{\text{ref}}$ as $t \rightarrow \infty$, due to the control integral term. Note that if the setpoint change is slower than the controller's time constant, then the vehicle will follow the desired time-changing speed setpoint. This plays a crucial role in Section 5.2 since we will use a smooth (i.e., differentiable) and variable setpoint that depends on curvature.

In our case, the rover measures its position directly from the global positioning system (GPS), discarding data from inertial sensors. Our rovers are small outdoor vehicles subject to bumpy and stony terrain and several other factors. This bounciness affects the accelerometers of the IMU, adding a significant amount of noise, almost equal to or larger than the actual acceleration of the rover due to displacement. Besides, the rover's acceleration transient time is faster than the noise perturbations (due to its relatively small mass). Therefore, the acceleration due to an actual increase or decrease in speed could not be differentiated from the noise signal. In addition, a proper measure of the noise process variance is complex since it depends on the terrain.

Thus, the GPS speed is critical, but this signal is also noisy. Filtering is needed to allow for the operation of a speed controller (especially the derivative controller). For this purpose, we use a simple *moving average* filter. Therefore, our rovers measure the speed using only the GPS signal filtered with a *moving average* filter, and they use this filtered speed measurement v as an input to the speed controller in Equation (29).

5.2 | Changing Speed Setpoint

Typically, path-following algorithms consider a constant speed when following a specific curve. This speed is chosen so that the vehicle can follow the trajectory with sufficiently small (guidance) control inputs. Moreover, their heading cannot change instantly when dealing with nonholonomic vehicles like the model in Equation (1). This means that the radius of curvature that it can trace increases (respectively, decreases) as speed increases (decreases). Therefore, if the speed is held constant, it must be restricted to the maximum value that ensures the rover can follow the trajectory. That is, the speed of the rover must allow its guidance controller to converge to the trajectory, regardless of high curvature segments. That is because guidance control signals (i.e., inputs to the steering angle) at relatively high speeds can cause a rollover. Moreover, the saturation of the steering angle creates the same problems explained above.

Besides, there are applications in which a constant speed is not desirable or interesting. For example, if the trajectory must be completed below a certain time stamp, an interesting approach is that the vehicle accelerates, where the curvature is low or zero (i.e., straight lines) and reduces its speed when curvature increases. For this purpose, we propose that the speed setpoint

can be changed according to the path's curvature, following the next equation:

$$v_{\text{ref}}(\kappa) = (v_{\text{max}} - v_{\text{min}})\exp(-c_{\kappa}\kappa^2) + v_{\text{min}}, \quad (32)$$

where v_{max} and $v_{\text{min}} \in \mathbb{R}_{\geq 0}$ are the maximal and minimal rover speed ($v_{\text{max}} \geq v_{\text{min}}$), $\kappa \in \mathbb{R}$ is the curvature and $c_{\kappa} \in \mathbb{R}_{\geq 0}$ is a constant used to penalize the speed as curvature increases. Since we are dealing with parametric curves, the curvature formula is well known (Dubrovnik et al. 1984) and is defined as

$$\kappa(w) = \frac{f'_1(w)f''_2(w) - f''_1(w)f'_2(w)}{(f'_1(w)^2 + f'_2(w)^2)^{3/2}}. \quad (33)$$

Thus, the curvature is computed using the parameter of the path w , that is, Equation (33) calculates the curvature of the point at which the VF is pointing to (see Figure 7).

Assuming that c_{κ} is greater than zero in Equation (32), we can observe that the speed setpoint, which will be utilized in the controller (29), is limited by $v_{\text{ref}} \in [v_{\text{min}}, v_{\text{max}}]$. The maximum value of the setpoint is achieved when $\kappa \rightarrow 0$, while the minimum value is obtained when $\kappa \rightarrow \infty$. By adjusting c_{κ} appropriately, the speed of the rover can be slowed down when it approaches areas with relatively high curvature. It is interesting to note that the exponential argument is quadratic. This solves two problems: (1) κ can take negative values depending on the sign of the curvature and (2) taking κ^2 instead of $|\kappa|$ makes the setpoint differentiable, providing the curve has at least C^3 continuity to have a differentiable curvature, which is a fine property if, for example, a derivative or nonlinear controller is desired.

In summary, the rover will reach its maximum speed whenever it is following a straight path, $\kappa = 0$; it will get a stable speed $v_{\text{max}} > v \geq v_{\text{min}}$ if it is following a curve of constant κ , that is, a circle or circle section and it will accelerate or decelerate if κ is decreasing or increasing. Parameters in Equation (32) allow us to tune the WMR behavior to the path and terrain features.

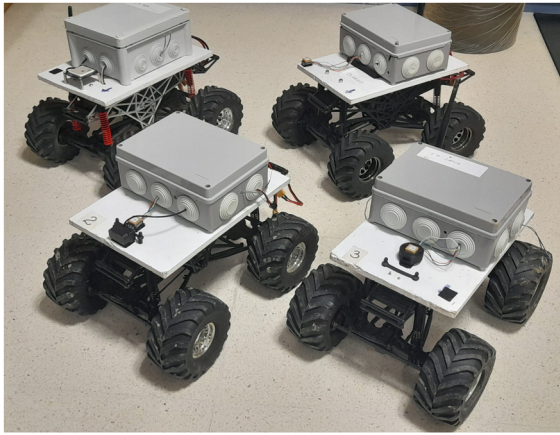
6 | Results

6.1 | Experimental Platform

We use the same software for simulations and experiments. In this way, the algorithms implemented for the simulation can also be used in the experiments without any further change. This section briefly describes the hardware and software platforms used for simulations and experiments.

6.1.1 | Rover's Hardware

Figure 4 shows several of our rovers. These rovers are commercial RC electric cars modified to provide a robust mechanical structure for their use as a testing platform, not only for algorithms designed for these vehicles but also for more complex systems, such as Unmanned Surface Vehicles (e.g., a boat whose direction is



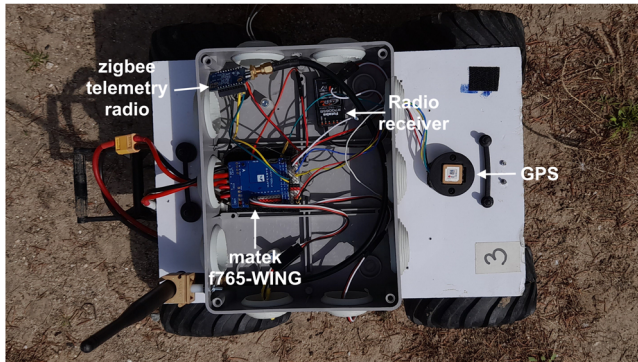
(a) View of the team of Rovers.



(b) Overview of the mechanical system of the Rovers.

FIGURE 4 | Rovers. (a) View of the team of rovers and (b) overview of the mechanical system of the rovers.**TABLE 1** | Rover's physical measurements. MSA stands for maximal steering angle.

Wheelbase (cm)	Axle track (cm)	Length (cm)	Width (cm)	Height (cm)	MSA outer wheel (°)	MSA inner wheel (°)
25	23	40	30	29	15	10

**FIGURE 5** | Rover's electronics hardware setup in an outdoors experiment. GPS, global positioning system.

controlled by a rudder or a differential drive vessel) or fixed-wing planes. The dimensions of the rover are collected in Table 1. We have removed the commercial radio receiver from the rovers and mounted on top of it a polyvinyl chloride foam board and a potting box, as shown in Figure 5. These rovers have a front-wheel Ackerman-type steering, which mechanically coordinates the angle of the two front wheels mounted on a standard axle for steering. Moreover, they are all-wheel drive and equipped with differential gears in both axes, reducing the skidding when turning since it allows each wheel (in each axis) to rotate at a different speed. The rover's movement is controlled by two motors: (1) an HPI Racing Saturn brushed direct current (DC) motor that provides thrust to all wheels and (2) a servo motor that drives the steering. The onboard autopilot is a Matek f765-Wing flight controller whose Microcontroller Unit is the STM32F765VIT6 from ST Microelectronics. Although it has been designed to control fixed-wing drones, the autopilot suits our purposes well, supplying

a common platform for different types of autonomous vehicles. In addition to a built-in Inertial Measurement Unit (IMU) in the flight controller, the rovers have several devices connected to it: (1) an ublox GPS (SAM-M8Q) receiver with a built-in compass, (2) a Digi Xbee S2C RF transceiver that allows both monitoring the rover's parameters and sending control signals to them from a ground station (see below), and (3) a Futaba RC receiver with SBus protocol, that allows the rovers to be remotely controlled using an RC Futaba radio and transfer the control to and from the autopilot. These devices also grant access to the necessary states: the position (p_x, p_y) and attitude θ , relative to the inertial frame $W(p)$, needed for the guidance controller shown in Equation (22), as well as access to the speed v , used (after filtering) by the speed controller in Equation (29). A 7.4 V Li-Po Battery powers the complete system with a capacity of 3000 mAh (Table 2).

6.1.2 | Paparazzi UAV Environment

As noted in Section 1, we use paparazzi as a development environment to program autonomous vehicles. Paparazzi is a free and open-source hardware and software project intended to create a flexible autopilot system. Although it was initially designed to deal with UAVs, researchers from different universities have extended its applications to other autonomous vehicles. Moreover, the paparazzi supports multiple hardware designs. To do so, it includes (1) a cross compiler, allowing the user to compile code for any supported flight controller board, (2) an uploader tool to upload the implemented code to the flight controller, and (3) support for several sensors and devices. The reader interested may address to paparazzi wiki pages: https://wiki.paparazziuav.org/wiki/Main_Page.

Paparazzi also provides a simulator with different simulation vehicle models (e.g., the rover's model, fixed-wing aircraft

TABLE 2 | Rover's hardware components.

Brushed DC motor	HPI racing saturn	Provides thrust to all wheels
Servo motor		Drives the steering
Microcontroller unit	STM32F765VIT6 (ST Microcontrollers)	Onboard Matek f765-Wing autopilot
Inertial measurement unit	MPU-6000 and ICM20602 (InvenSense)	Onboard Matek f765-Wing autopilot
GPS	ublox GPS (SAM-M8Q)	Provides speed and positioning
RF transceiver	Digi Xbee S2C RF	Provides communication to ground station
Radio receiver controller	RC Futaba with SBus protocol	Allows one to remote control the rover
Battery	7.4 V Li-Po 3000 mAh	Rover power

Abbreviations: DC, direct current; GPS, global positioning system; HPI, Hobby Products International; RC, radio control; RF, radio frequency.

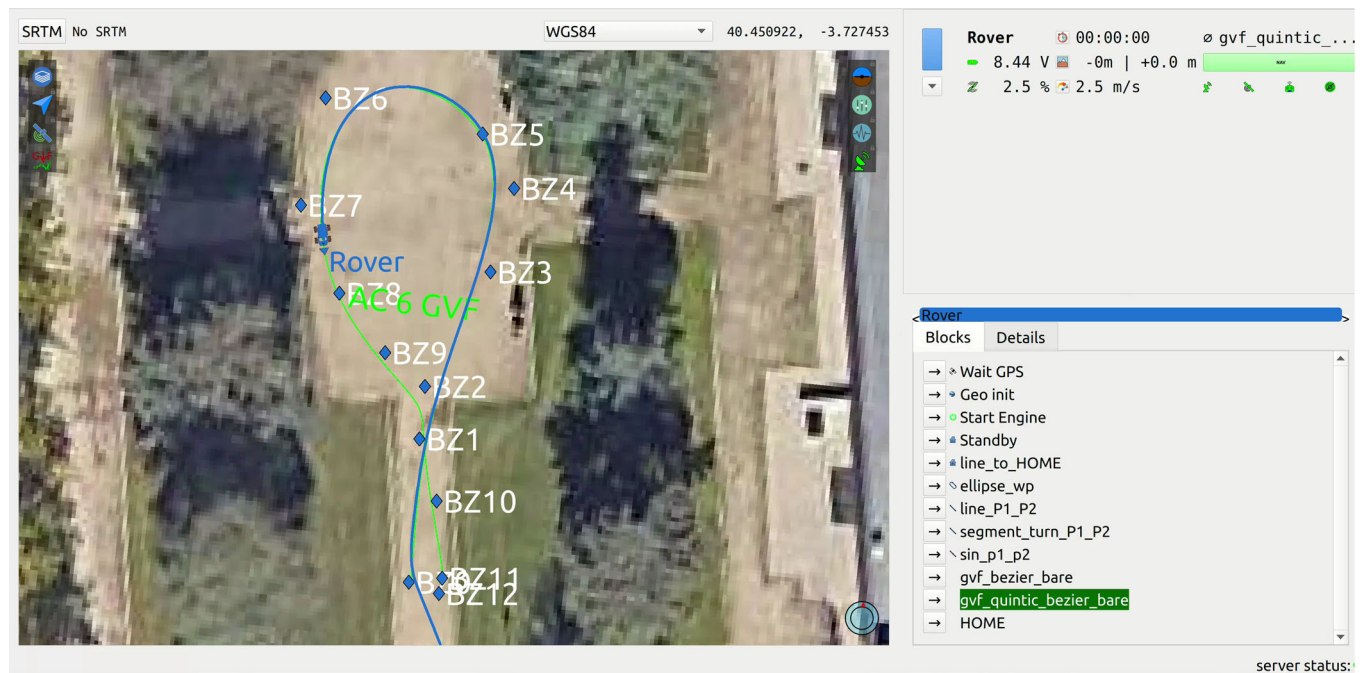


FIGURE 6 | Screenshot from the paparazzi's GCS in simulation. Bézier points are represented as blue diamonds, while the desired trajectory is represented as a green line. The actual trajectory traced by the rover is shown in blue. GCS, Ground Control Station; GPS, global positioning system; GVF, guiding vector field; SRTM, Shuttle Radar Topography Mission.

models, and rotorcraft models) as well as sensor models. Besides, it is possible to modify the models or add new ones to fit the model to a specific AMR. The simulator is intended to test the performance of new algorithms before implementing them in the actual vehicles, saving time and resources.

The third and last component of paparazzi is a Ground Control Station (GCS), shown in Figures 6 and 7, where agents' positions are visualized, and different sensor and internal variables can be monitored. The paparazzi GCS is coded in C++ and can be customized to include new properties. For instance, we have added the code to define, show, and modify the Bézier curves used for path planning on the fly. To offer maximum flexibility and openness, the paparazzi ecosystem was designed from the start as a modular one (Gati and Balazs 2013). The user can specify the type of vehicle and sensors used, the flight plan, and the desired telemetry data using Extensible Markup Language (XML) files. This allows the user to choose between several XML modules without developing any C-code. Therefore,

thanks to the modular software architecture, generating the basic navigation code for the rovers has been accelerated by using the different modules available in the paparazzi framework, such as GPS, radio control, wireless communication, and others.

Figure 8 shows a block diagram with the system's main features. Using the paparazzi GCS, the user can give real-time commands as inputs to the rover's autopilot. Examples of inputs include path-following commands of a desired curve—waypoints, speed setpoints, and guidance and speed controller constants. The rest of the system runs in the rover's flight controller, allowing for a fully autonomous mode. Concerning the autonomous mode, states (p_x, p_y) and speed v are obtained from GPS data and sent to the SF-GVF module. This module computes the desired trajectory $f(w)$, the vector field $\hat{\chi}^p$ and the term $\hat{e}_d$ needed for u_θ , as well as w and $\kappa(w)$ required in the speed controller. The heading and speed controllers generate the control signals $u_\theta \rightarrow \phi$ and u_v by using the states,

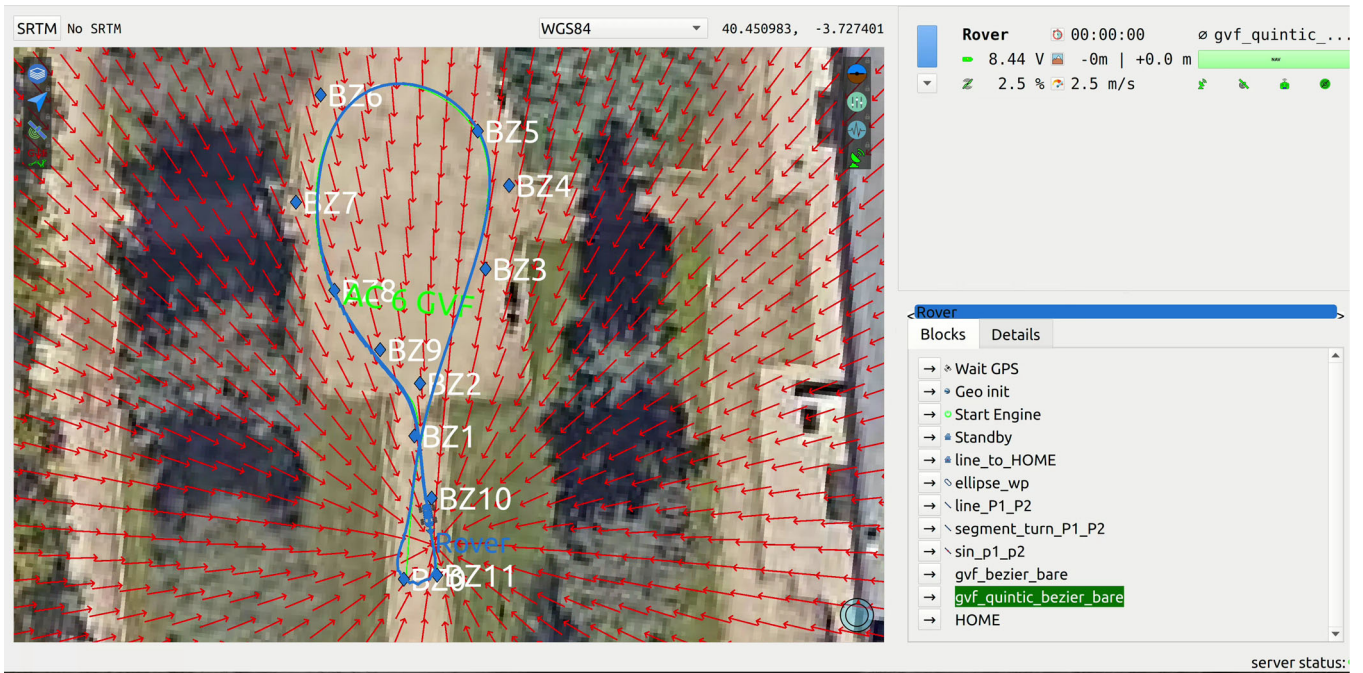


FIGURE 7 | Screenshot from the same simulation as Figure 6 but forwarded in time and showing the vector field $\hat{\chi}^p$ as red arrows. GPS, global positioning system; GVF, guiding vector field; SRTM, Shuttle Radar Topography Mission.

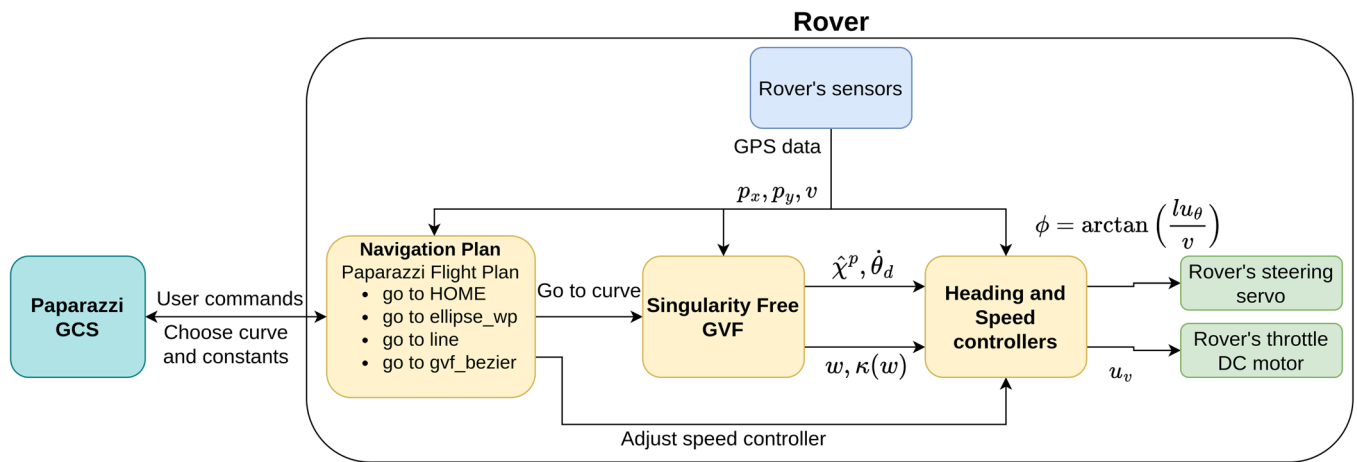


FIGURE 8 | Schematic representation of the system. Once the code has been uploaded to the AMR from paparazzi, GCS commands can be given to the rover via telemetry. Then, the rover follows the trajectory and speed setpoint according to the modules shown in the figure. AMR, autonomous mobile robot; DC, direct current; GCS, Ground Control Station; GPS, global positioning system; GVF, guiding vector field.

guidance information from the SF-GVF, and controller constants given by the user via paparazzi GCS. Moreover, the states are fed to the Navigation Plan block to show these variables in real-time in paparazzi GCS. Finally, the control signals are sent to the lowest level of the paparazzi UAV (not shown in the figure; it maps the values of ϕ and u_v to PWM signals) to turn the rover's steering servo and spin the DC motor.

The heading and speed controllers have been implemented in paparazzi (C-code), allowing us to use them in simulation and in the experimental platform. First, the algorithm in Section 3 has been modified to be used with rover vehicles since it was already implemented in the paparazzi for fixed-wing aircraft.

Moreover, the capability of following Bézier curves has been added. Thus, the user can not only create a desired Bézier curve in the paparazzi GCS but modify it when desired (e.g., during an experiment) since it is the flight controller in the rover that automatically computes the Bézier curve control points for continuity conditions. Second, the speed controller of Equation (29) has been implemented using the modules paparazzi provides. Third, the changing speed setpoint of Section 5 and a moving average filter for the measured speed have been implemented.

Finally, to see in real-time the desired parametric curve $f(w)$, the authors have implemented the code to show the trajectory and also the varying vector field $\hat{\chi}^p$, as shown in Figures 6 and 7

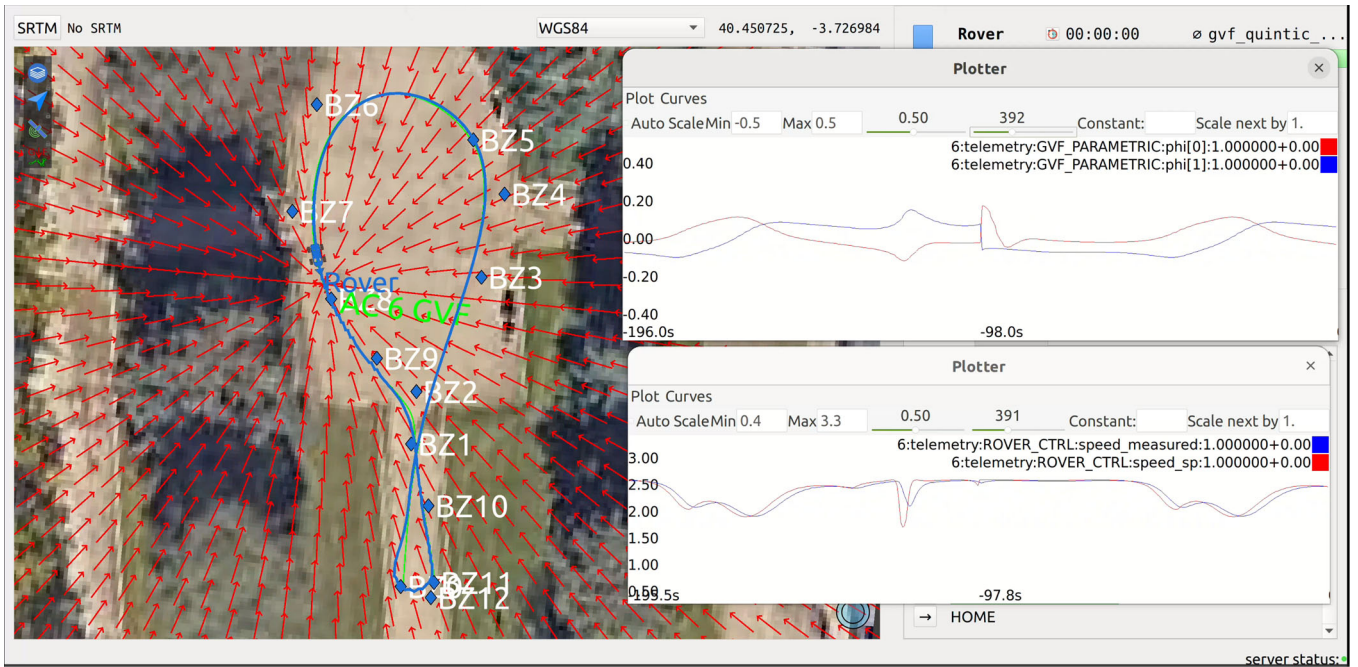


FIGURE 9 | Screenshot from the paparazzi's GCS in simulation. Upper right graphic shows the distance errors in meters (ϕ_1, ϕ_2) to the desired path $\mathcal{P}$ (as $\phi_i[0]$ and $\phi_i[1]$), respectively versus time. The bottom right graphic shows the time evolution of v_{ref} and v as red and blue lines in m/s. Constants $k_1 = k_2 = 0.5$ m, $k_\theta = 3$ for the guidance controller and $v_{min} = 1.7$ m/s, $v_{max} = 2.7$ m/s, $c_k = 10$ m² for the speed controller were used. GCS, Ground Control Station; GVF, guiding vector field; SRTM, Shuttle Radar Topography Mission.

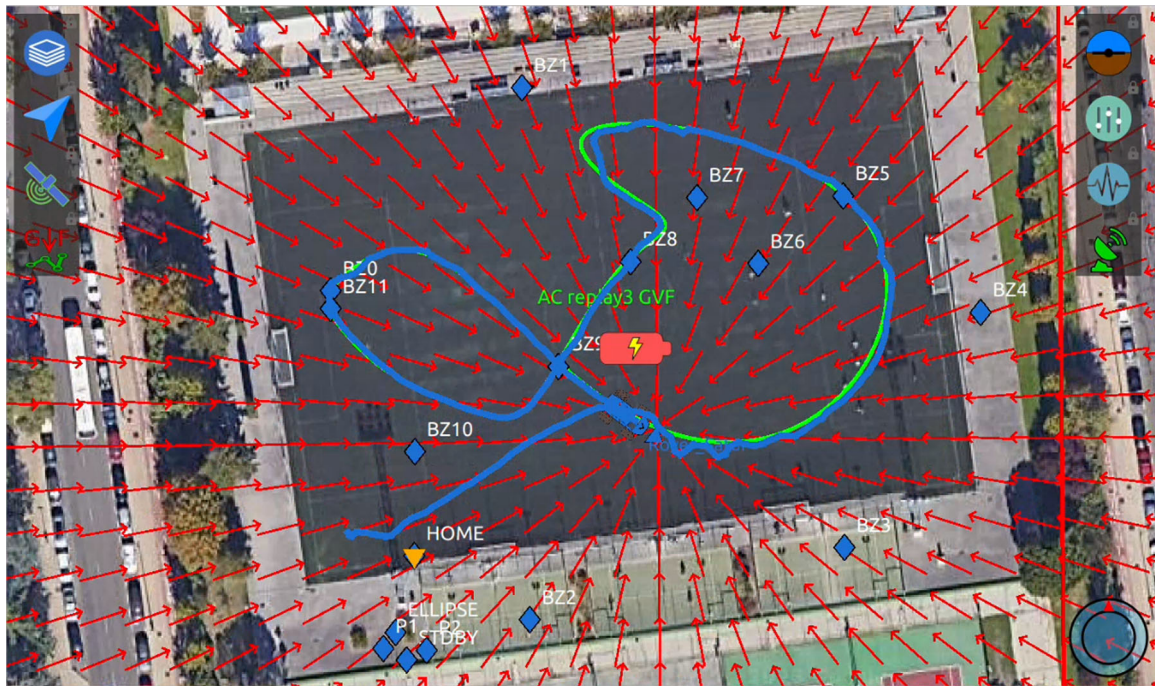


FIGURE 10 | Screenshot from the paparazzi's GCS in the first real experiment. The representations and meanings are the same as in the simulation. The same constants were used as in the simulation. AC, alternating current; GCS, Ground Control Station; GVF, guiding vector field.

(where a fifth-order Bézier curve with C^2 continuity is shown). This allows us to adjust the guidance constants (i.e., k_1 , k_2 , and k_θ) by visualizing the deviations from the desired trajectory and the point the VF is pointing to.

It is important to note that waypoints can be adjusted to fit the trajectory into the desired area. By simply moving the desired control or endpoints, the user can shape the path as they wish. Onboard software recalculates the trajectory whenever a point



FIGURE 11 | Screenshot from the paparazzi's GCS in the second real experiment. The same constants were used as in the simulation. AC, alternating current; GCS, Ground Control Station; GVS, guiding vector field.

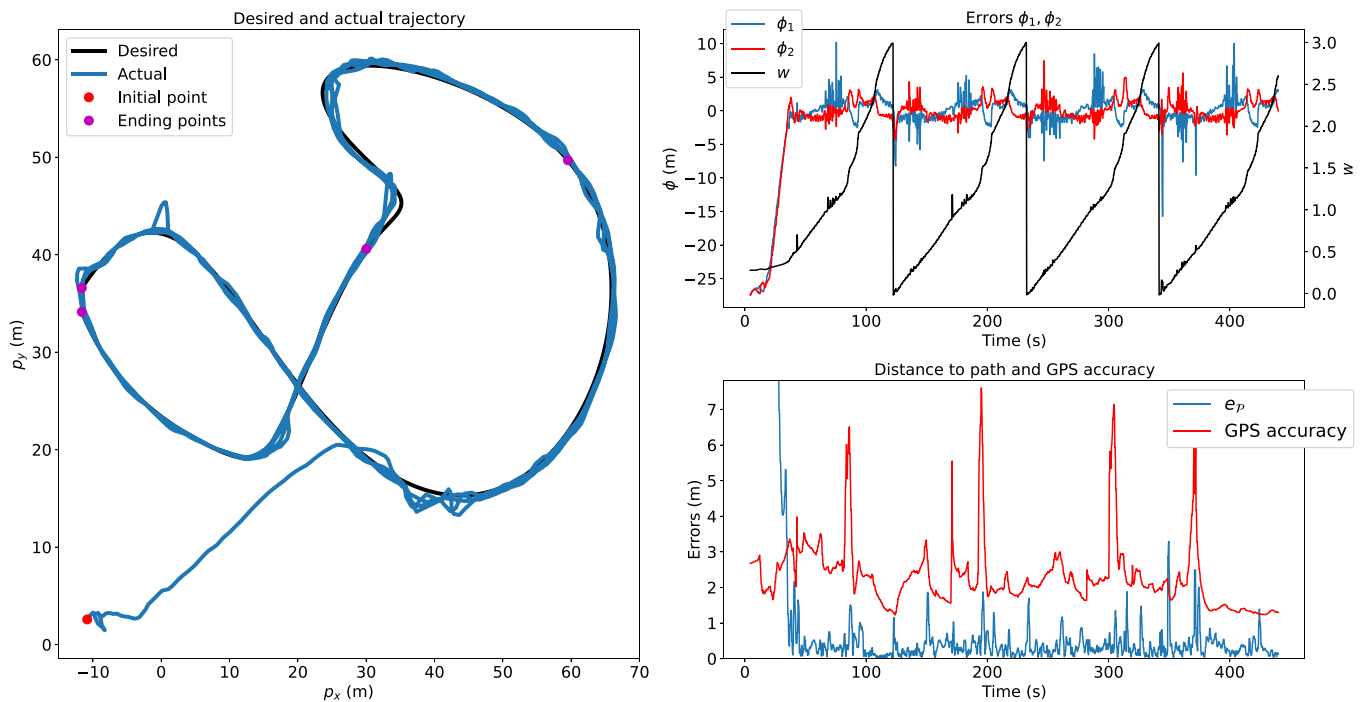


FIGURE 12 | Experimental data obtained from the logs for a complete set of trajectories traced by the rover. The left subfigure shows the actual traced trajectory in blue, the desired one in black, the initial point in red, and the curve points $\beta_0, \beta_5, \beta_8, \beta_{11}$ in magenta. The upper right subfigure shows the errors ϕ_1, ϕ_2 in the left y-axis and the parameter w in the right y-axis. The bottom right subfigure shows the distance to the path $e_p = \inf\{\|p - p_d\| \mid p_d \in \mathcal{P}\}$ and the accuracy of the GPS position. GPS, global positioning system.

has changed. This allows for considerable flexibility in experimental environments where uncertainties in position may arise.

The main code for following third-order Bézier curves is under the central paparazzi repository: <https://github.com/paparazzi/>

paparazzi, for following both third- and fifth-order Bézier curves is under the repository https://github.com/alfredoFBW/paparazzi_alfredoFBW, while the code to visualize both Bézier curves and the VF in paparazzi GCS is under <https://github.com/alfredoFBW/PprzGCS>.

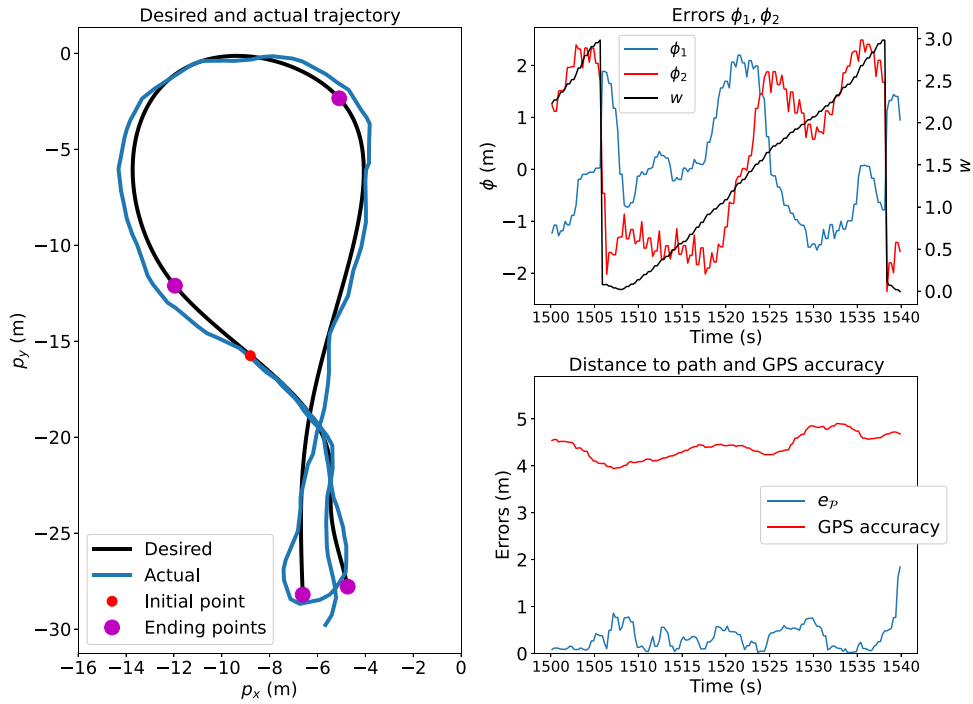


FIGURE 13 | Experimental data obtained from the logs. The left subfigure shows a set of actual trajectories and the desired one, the initial point in red, and the curve points $\beta_0, \beta_5, \beta_8, \beta_{11}$ in magenta, the upper right subfigure shows in the left y-axis the errors ϕ_1, ϕ_2 , and in the right y-axis the parameter w . The bottom right subfigure shows the distance to the path $e_p = \inf\{\|p - p_d\| | p_d \in \mathcal{P}\}$, and the GPS position accuracy. GPS, global positioning system.

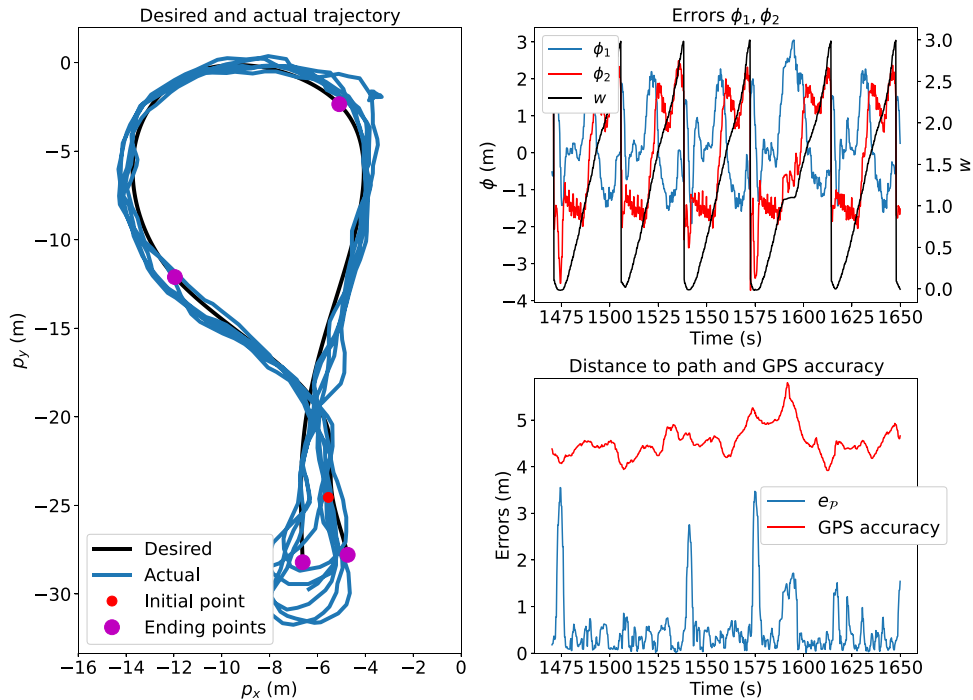


FIGURE 14 | Experimental data obtained from the logs of the same experiment, but of a complete set of trajectories. The left subfigure shows a set of actual trajectories and the desired one, the initial point in red, and the curve points $\beta_0, \beta_5, \beta_8, \beta_{11}$ in magenta, the upper right subfigure shows in the left y-axis the errors ϕ_1, ϕ_2 , and in the right y-axis the parameter w . The bottom right subfigure shows the distance to the path $e_p = \inf\{\|p - p_d\| | p_d \in \mathcal{P}\}$, and the GPS position accuracy. GPS, global positioning system.

6.2 | Simulation Results

A trajectory is created to test the SF-GVF and heading controller in simulation using $N = 3$ fifth-order ($n = 5$) Bézier curves. As explained in Section 4, a degree fifth Bézier curve is defined by $n + 1 = 6$ points: the starting and end points and four control points, which can be used to shape the curve. Since the spline has C^2 continuity, the endpoint of the segment i is the starting point of segment $i + 1$, $i \in [0, N - 2]$, and the two control points β_1^i, β_2^i of segment i came defined by the values of the second and third to last points of the previous segment $\beta_4^{i-1}, \beta_3^{i-1}$, giving us two control points to shape each spline segment. Thus, we have $3(N + 1) = 12$ configurable points for the complete spline, which we define as β_k^s . In particular, $\beta_k^s : k \in \{0, 5, 8, 11\}$ define each segment's starting and end points, while the rest are used as control points to modify each segment's shape independently. Therefore, the curve starts at β_0^s and ends at $\beta_{3N-2}^s = \beta_{11}^s$. An example of a curve is created and shown in Figure 9, where the trajectory intersects with itself and has a changing curvature, the necessary conditions to test our controllers. After launching the simulation using the model from Equation (3), it can be seen how the path traced by the rover converges to the desired parametric curve and errors ϕ_1, ϕ_2 are in the order of centimetres. Moreover, it can be seen how the speed setpoint (speed_sp) and the rover's speed (speed measured) change according to the path's curvature. Note that the algorithm is restarted when reaching the ending point β_{11} by resetting the path parameter w to the initial value $w = 0$.

6.3 | Real Rover Results

This subsection is divided into two parts. The first deals with the actual experiments on the guidance algorithm, while the second deals with the speed control problem. It is also important to note that the following experiments can be reproduced by compiling and downloading the code to the vehicle (provided they have the same or compatible hardware) using the repositories presented above.

Two self-intersecting curves are chosen for the SF-GVF and heading controller experimental tests. The first one is an experimental setup with fair environmental conditions, that is, a clear sky for the GPS receiver and an obstacle-free football field. The second one represents a worst-case scenario, where the area in which the rover can move is tighter, and the GPS is not providing the best possible resolution. Now, the speed setpoint is the same for both experiments and is implemented as in Equation (32) using $v_{\min} = 1.4\text{m/s}$, $v_{\max} = 2.4\text{m/s}$, $c_k = 15\text{m}^2$. Using the paparazzi replay utility, real data results from both experiments are reproduced in Figure 10 for the first case and Figure 11 for the second one.

Paparazzi replay utility tool allows us to select time intervals of interest to extract data for their analysis. Besides, paparazzi logs the vehicle's status with the Zigbee telemetry radio. Hence, data from the experiments can be extracted for its posterior analysis. Thus, data from the logs were extracted for both curves to

determine the magnitude of the errors. These data are displayed for a complete set of trajectories traced by the rover for the first experiment curve in Figure 12, and for the second one in Figures 13 and 14. Figure 13 shows a particular trajectory of the second experiment, while Figure 14 shows a set of trajectories made during the same experiment.

For the first experiment, in Figure 12, the left graphic represents the actual trajectory as a blue line and the desired trajectory as a black line, where the origin of coordinates is centered on the HOME point shown in Figure 10. Thus, the rover position (p_x, p_y) is measured relative to the HOME origin attached to the earth and fixed throughout the whole experiment. Home location is determined by the first geographical position the GPS measures.

The rover begins 25 m away from the intended path (as shown on the left side graphic of Figure 12). The algorithm then guides the rover towards the path until it reaches it. Once the final segment of the path is reached, the trajectory begins again, the algorithm restarts, and w is reset to 0. This is indicated by the rover converging back to the starting point of the path. The upper right graphic shows the evolution of the errors ϕ_1 and ϕ_2 (on the left y-axis) and the parameter w (on the right y-axis) over time. The bottom left graphic displays the error to the path, $e_p = \inf\{\|p - p_d\| | p_d \in \mathcal{P}\}$, as well as the GPS position accuracy.

Once the rover has converged to the trajectory, the distance to the path e_p is close to zero and well below the GPS position accuracy throughout the experiment. Note, however, that the errors ϕ_1, ϕ_2 , and thus the previously mentioned error $\|e(\xi)\|$ with $e(\xi) = (\phi_1(\xi), \phi_2(\xi))$, are bigger, and in certain time instants they can reach errors of 10 m. This behavior is due to oscillations in the parameter w , whose dynamics were specified in Equation (18).

As can be seen in the upper right graphic, the time instants in which w oscillates match those times when the errors ϕ_1, ϕ_2 oscillate and separate from zero. Recall that $\phi_1 = p_x - f_1(w)$ and $\phi_2 = p_y - f_2(w)$ and, for a three-segment Bézier spline, $w \in [0, 3]$. Thus, even if the rover has converged to the trajectory, a fast oscillation of w creates fast oscillations of $f_1(w)$ and $f_2(w)$, which in turn creates fast oscillations of ϕ_1 and ϕ_2 . Moreover, since $w \in [0, 3]$ and the dimensions of the Bézier spline are in the order of tens of meters, a fast and small variation of w can create a big variation in $f(w)$ and a big and fast transient in ϕ_1 and ϕ_2 . Nevertheless, the distance to the path, e_p , is not directly affected by w , and from the left graphic and the bottom right graphic, it can be seen that it converges towards zero and remains below the GPS accuracy.

For the second experiment, Figure 14 represents the same information as Figure 13 but for a set of trajectories. Note that, compared to Figure 12, the area in which the rover must remain is tighter, as can be seen from the dimensions of the trajectory. Moreover, the mean GPS accuracy is close to the one in Figure 12, allowing for a similar error if the GPS accuracy is considered as the bound of the d term of Equation (24). As

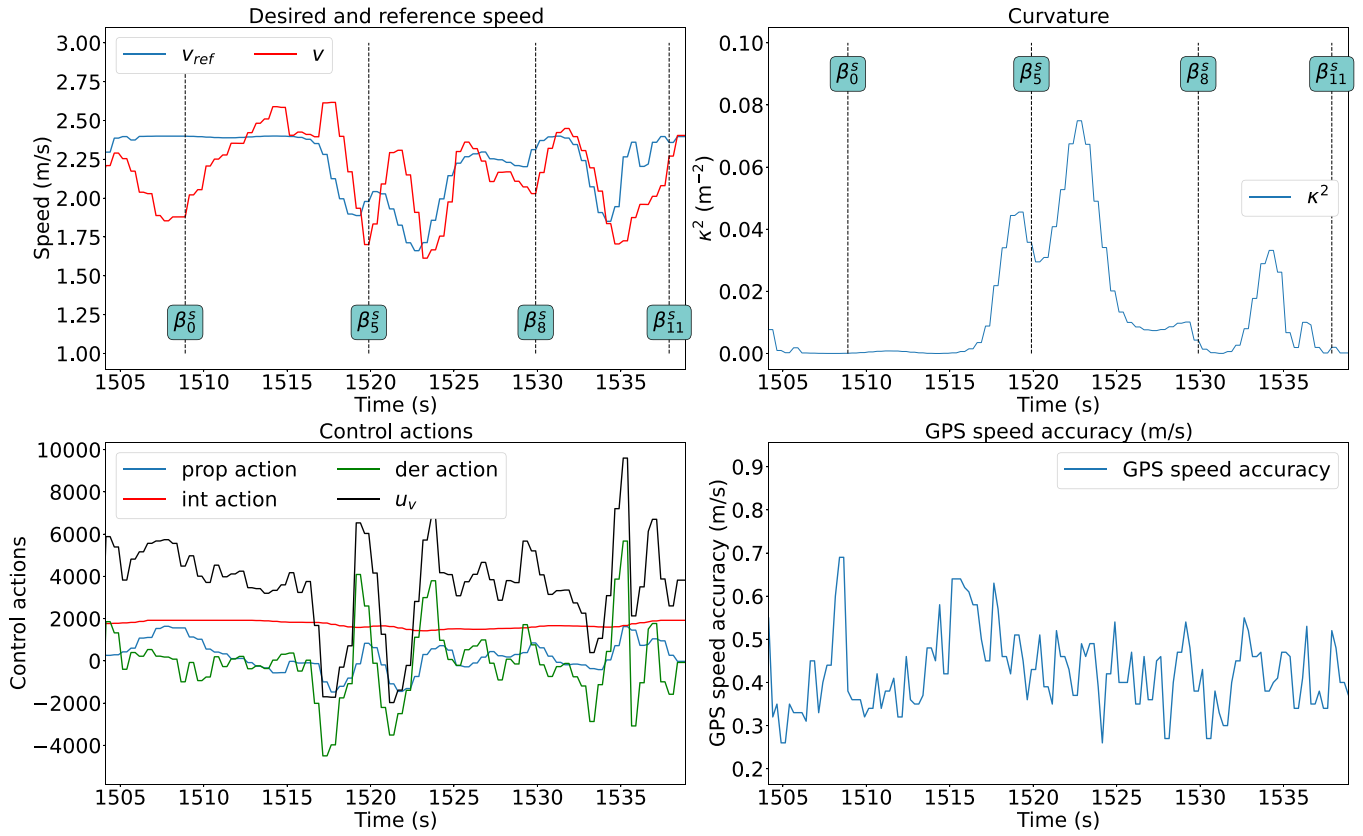


FIGURE 15 | Rover's speed data obtained from the same experiment as in Figure 13. The top left graphic shows the desired speed setpoint as a blue line, the rover's measured speed as a red line and as vertical black dotted lines the timestamps in which the rover passes through the corresponding Bézier points β_i^s . The top right graphic represents the curvature as a function of time $\kappa(w(t))$. The bottom left graphic represents the proportional, integral, and derivative control actions and the total speed control action u_v . The bottom right graphic represents the speed accuracy given by the GPS. Constant used for this experiment: $v_{\min} = 1.4\text{ m/s}$, $v_{\max} = 2.4\text{ m/s}$, $c_\kappa = 15\text{ m}^2$, $k_f = 1000$, $k_p = 3000$, $k_i = 300$, and $k_d = 2000$. GPS, global positioning system.

TABLE 3 | Bézier control points measured from HOME position and constants used for the first experiment.

Bézier points used for the first experiment				Controller constants	
Bézier points (w.r.t. HOME)	Segment 0 (m)	Segment 1 (m)	Segment 2 (m)	Constant used	Values
β_0^i	(−11.62, 36.58)	(59.53, 49.69)	(30.12, 40.59)	k_1, k_2	0.5
β_1^i	(14.93, 64.67)	(40.45, 65.78)	(20.78, 31.71)	k_θ	1
β_2^i	(16.02, −8.84)	(−16.64, 65.54)	(10.78, 4.84)	k_f	1000
β_3^i	(59.72, 1.15)	(47.74, 40.36)	(20.00, 26.13)	(k_p, k_i, k_d)	(3000, 300, 2000)
β_4^i	(78.63, 33.59)	(39.26, 49.47)	(0.07, 14.38)	c_κ	15
β_5^i	(59.54, 49.69)	(30.02, 40.59)	(−11.63, 34.13)	M	200

explained above, the algorithm restarts when reaching the end point of the spline, which means that the trajectories shown in Figure 14 are done sequentially, that is, in the same experiment and one after another.

In this more extensive set of experiments, similar to the previous one, the error is bounded from above throughout the entire experiment by the GPS position accuracy; as it is reflected in the comparison between the desired and actual trajectory. Moreover, at the starting points, it can be seen that a transient loop

occurs. This is due to the asymptotic stability of the closed loop system; when the curve is restarted, it shall converge to the trajectory again. Moreover, compared with Figure 12, this experiment did not show any fast oscillations of the parameter w , and in turn, no fast oscillations of the errors ϕ_1 and ϕ_2 .

For completeness, let us use the results of Equation (27), where we arrived at the result that $\|e(\xi)\| \leq \sup_{t \in [0, \infty)} \frac{\|d(t)\|}{\sqrt{\lambda_{\min}}}$. The

Q matrix can be expressed as

TABLE 4 | Bézier control points measured from HOME position and constants used for the second experiment.

Bézier points used for the second experiment				Controller constants	
Bézier points (w.r.t. HOME)	Segment 0 (m)	Segment 1 (m)	Segment 2 (m)	Constant used	Values
β_0^i	(−6.61, −28.20)	(−5.08, −2.34)	(−11.95, −12.10)	k_1, k_2	0.5
β_1^i	(−6.85, −20.01)	(−7.20, 0.55)	(−8.89, −16.75)	k_θ	1
β_2^i	(−6.86, −17.00)	(−12.26, −1.49)	(−2.00, −19.81)	k_f	1000
β_3^i	(−3.80, −10.11)	(−14.25, −1.25)	(−6.86, −22.65)	(k_p, k_i, k_d)	(3000, 300, 2000)
β_4^i	(−2.97, −5.25)	(−15.01, −7.46)	(−5.51, −23.45)	c_κ	15
β_5^i	(−5.08, −2.34)	(−11.95, −12.10)	(−4.73, −27.78)	M	200

$$Q = \begin{bmatrix} q_{11} & q_{12} \\ q_{12} & q_{22} \end{bmatrix} = \begin{bmatrix} (f_1'(w)^2 + 1)k_1^2 & f_1'(w)f_2'(w)k_1k_2 \\ f_1'(w)f_2'(w)k_1k_2 & (f_2'(w)^2 + 1)k_2^2 \end{bmatrix} > 0.$$

The eigenvalues can be computed to obtain $\lambda = \frac{q_{11} + q_{22} \pm \sqrt{(q_{11} + q_{22})^2 - 4(q_{11}q_{22} - q_{12}^2)}}{2}$. Moreover, note that in both experiments, we have used the same values of k_1 and k_2 , that is, $k_1 = k_2 = 0.5$, so plugin in this condition on the eigenvalues and the values of the Q matrix, we arrive at

$$\lambda = \frac{k_1^2(f_1'(w)^2 + f_2'(w)^2 + 2) \pm \sqrt{k_1^4(f_1'(w)^2 + f_2'(w)^2)^2}}{2}$$

$$= \frac{k_1^2(f_1'(w)^2 + f_2'(w)^2 + 2) \pm k_1^2(f_1'(w)^2 + f_2'(w)^2)}{2}.$$

Thus, it can be easily seen that $\lambda_{\min} = k_1^2, \lambda_{\max} = k_1^2(\|f'(w)\|^2 + 1)$. So the error can now be bounded, $\|e(\xi)\| \leq \sup_{t \in [0, \infty)} \frac{\|d(t)\|}{\sqrt{\lambda_{\min}}} = \sup_{t \in [0, \infty)} \frac{\|d(t)\|}{\sqrt{k_1^2}}$, so $\|e(\xi)\| \leq 2 \sup_{t \in [0, \infty)} \|d(t)\|$. For the first experiment $\sup \|d(t)\| = 7.6$, while for the second $\sup \|d(t)\| = 5.3$, so for the first experiment $\|e(\xi)\| \leq 15.2$ m, while for the second $\|e(\xi)\| \leq 10.6$ m. This holds true in both experiments when computing $\|e(\xi)\| = \sqrt{\phi_1(\xi)^2 + \phi_2(\xi)^2}$, which connect theoretical and experimental results. Note that in the first experiment, the supremum of the disturbance is larger (i.e., there are instants in which the GPS uncertainty is larger), while the average GPS position error is lower than in the second experiment, as can be seen by comparing Figure 12 with Figure 13 or 14.

Before commenting on the results of the speed controller, it is worth noting that in the paparazzi platform, the throttle control actions u_v are bounded by $u_v \in [-9600, 9600]$, and those values are converted to PWM signals as explained above.

Speed controller data from the second set of experiments is shown in Figure 15. The top left graphic shows the speed setpoint v_{ref} and the GPS speed v filtered with a moving average filter (a low pass filter) of $M = 200$ samples. Vertical black dotted lines represent the timestamps in which the rover passes through the corresponding Bézier points β_i^s . The top right

graphic depicts the curvature of the Bézier spline. As it can be seen from both top graphics, the speed setpoint v_{ref} changes accordingly to the computed curvature κ , reducing the setpoint when κ^2 increases and vice versa. The bottom left graphic shows the speed control signal u_v and its PID components, and the bottom right graphic represents the speed accuracy given by the GPS.

The time delay that can be seen in the top left graphic, where the actual speed v is behind the setpoint, is due to the speed controller settling time and the filter's effect. However, the delay in some way is not an undesired property. At any given time t , the VF points towards a point of the Bézier curve ahead of the rover, as shown in Figures 7 and 10. The speed setpoint changes based on the curvature of the function $f(w)$; however, assuming the rover has already reached the trajectory at time t , it will be at a point $f(w - w^*)$, where $w - w^* < w$. This indicates that the rover will reach the point $f(w)$, for which the setpoint was originally calculated after some delay. Thus, the trade-off between noise and time delay can be adjusted with the speed controller constants and the number of samples M . In particular, the constants we used are shown in Table 4. Finally, for completeness, Tables 3 and 4 show the Bézier points and constant values used in the first and second sets of experiments, respectively. Note that except for the first segment (segment $i = 0$), the points β_1^i, β_2^i for $i \in \{1, 2\}$ are computed using the C^2 continuity conditions shown in Section 4. Thus, the user only needs to specify the points in Figure 10.

7 | Conclusions

In this paper, a guidance system based on GVs is designed for path-following in AMR vehicles, and a curvature-dependent speed controller is used for faster convergence of the guidance algorithm. The definition of the desired path as parametric curves allows for a straightforward computation of the curve's curvature for the speed setpoint. In addition to presenting theoretical work involving these two controllers, simulations of the complete system are made using the paparazzi UAV platform. Finally, we introduce our AMR platform, for which exhaustive experiments are made to show the approach's effectiveness. The main contributions of this work are as follows:

- Generate a waypoint trajectory using Bézier curves, allowing the human user to easily set the waypoints and control points on the map and change them on the fly if necessary. Bézier curves allow smooth trajectories to be generated within the defined bounding box, formally called a convex hull.
- Once the trajectory is established, GVF is employed to define a VF that allows us to know, at any point in space, the desired velocity direction of the rover that will allow it to converge to the target path. The VF can be applied to derive a control law for the convergence property. GVF algorithms show exemplary performance. To the authors' knowledge, no results of this SF-GVF applied to the rover path-following problem exist.
- Using SF-GVF and the rover kinematic model, a controller is derived using a Lyapunov function, assuring stability.
- Path following is improved by speed control based on path curvature.
- Results of the guidance and speed controllers are presented in simulation and with a real rover in a natural environment. For this purpose, the paparazzi environment is used (Gati and Balazs 2013).

The ease and adaptability of Bézier polynomials make them very attractive to extend their use to other autonomous vehicles. For instance, we are currently testing the same approach to control Autonomous Surface Vehicles for water quality monitoring in inland reservoirs. Simulation results are presented in González-Calvin et al. (2023), using a planner and a guidance and control algorithm to traverse the planned trajectory. The results presented in this paper: generation of trajectories from measurement points using Bezier curves, tracking of these trajectories using SF-GVF and velocity control based on the curvature of the path are being tested on a USV with measurement probes for the detection of cyanobacterial blooms. The results are promising and will be the object of future publications.

Besides, we are also interested in extending our algorithm for obstacle avoidance, using data from proximity sensors to modify the shape of the Beziérs curves on the fly. So, the vehicle will replan its path, maintaining the prescribed waypoints wherever possible.

Acknowledgments

This work is partially supported by IA-GES-BLOOM-CM (Y2020/TCS-6420) of the Synergic Projects program of the Autonomous Community of Madrid and INSERTION (PID2021-27648OB-C33) of the Knowledge Generation program of the Ministry of Science and Innovation.

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

Bechar, A., and C. Vigneault. 2016. "Agricultural Robots for Field Operations: Concepts and Components." *Biosystems Engineering* 149: 94–111.

Bechar, A., and C. Vigneault. 2017. "Agricultural Robots for Field Operations. Part 2: Operations and Systems." *Biosystems Engineering* 153: 110–128.

Cao, Y., Z.-C. Wang, F. Liu, P. Li, and G. Xie. 2019. "Bio-Inspired Speed Curve Optimization and Sliding Mode Tracking Control for Subway Trains." *IEEE Transactions on Vehicular Technology* 68, no. 7: 6331–6342.

De Marina, H. G., Z. Sun, M. Bronz, and G. Hattenberger. 2017. "Circular Formation Control of Fixed-Wing UAVs With Constant Speeds." In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 5298–5303. IEEE.

Dubins, L. E. 1957. "On Curves of Minimal Length With a Constraint on Average Curvature, and With Prescribed Initial and Terminal Positions and Tangents." *American Journal of Mathematics* 79, no. 3: 497–516.

Dubrovin, B. A., A. T. Fomenko, and S. P. Novikov. 1984. "Modern Geometry—Methods and Applications." In *Part I. The Geometry of Surfaces, Transformation Groups, and Fields of Graduate Texts in Mathematics (GTM)*, edited by S. Axler, F. W. Gehring and K. A. Ribet, Vol. 93. Springer.

Faraway, J. J., M. P. Reed, and J. Wang. 2007. "Modelling Three-Dimensional Trajectories by Using Bézier Curves With Application to Hand Motion." *Journal of the Royal Statistical Society Series C: Applied Statistics* 56, no. 5: 571–585.

GámezSerna, C., and Y. Ruichek. 2017. "Dynamic Speed Adaptation for Path Tracking Based on Curvature Information and Speed Limits." *Sensors* 17, no. 6: 1383.

Gati and Balazs. 2013. "Open Source Autopilot for Academic Research—The Paparazzi System." In *2013 American Control Conference*, 1478–1481. IEEE.

González-Calvin, A., L. García-Pérez, J. L. Risco-Martín, and E. Besada-Portas. 2023. "Simulation, Optimization and Control of Trajectories of ASVs Performing HACBs Monitoring Missions in Lentic Waters." In *2023 Winter Simulation Conference (WSC)*, 910–921. Institute of Electrical and Electronics Engineers Inc.

Gu, N., D. Wang, Z. Peng, J. Wang, and Q.-L. Han. 2022. "Advances in Line-of-Sight Guidance for Path Following of Autonomous Marine Vehicles: An Overview." *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 53, no. 1: 12–28.

Han, L., H. Yashiro, H. T. N. Nejad, Q. H. Do, and S. Mita. 2010. "Bezier Curve Based Path Planning for Autonomous Vehicle in Urban Environment." In *2010 IEEE Intelligent Vehicles Symposium*, 1036–1042. IEEE.

Hansford, D. 2002. "Chapter 4—Bézier Techniques." In *Handbook of Computer Aided Geometric Design*, edited by G. Farin, J. Hoschek and M.-S. Kim, 75–109. North-Holland.

Hilario, L., N. Montés, M. C. Mora, and A. Falcó. 2011. "Real-Time Bézier Trajectory Deformation for Potential Fields Planning Methods." In *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 1567–1572. IEEE.

Hitz, G., E. Galceran, M.-È. Garneau, F. Pomerleau, and R. Siegwart. 2017. "Adaptive Continuous-Space Informative Path Planning for Online Environmental Monitoring." *Journal of Field Robotics* 34, no. 8: 1427–1449.

Hwang, J.-H., R. C. Arkin, and D.-S. Kwon. 2003. "Mobile Robots at Your Fingertip: Bezier Curve On-Line Trajectory Generation for Supervisory Control." In *Proceedings of the 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003) (Cat. No. 03CH37453)*, Vol. 2, 1444–1449. IEEE.

Jolly, K., R. S. Kumar, and R. Vijayakumar. 2009. "A Bezier Curve Based Path Planning in a Multi-Agent Robot Soccer System Without Violating the Acceleration Limits." *Robotics and Autonomous Systems* 57, no. 1: 23–33.

- Kapitanyuk, Y. A., H. G. de Marina, A. V. Proskurnikov, and M. Cao. 2017. "Guiding Vector Field Algorithm for a Moving Path Following Problem." *IFAC-PapersOnLine* 50, no. 1: 6983–6988.
- Kawabata, K., L. Ma, J. Xue, C. Zhu, and N. Zheng. 2015. "A Path Generation for Automated Vehicle Based on Bezier Curve and via-Points." *Robotics and Autonomous Systems* 74: 243–252.
- Lavalle, S. M. 2006. *Planning Algorithms*. Cambridge University Press.
- Leng, Z., and M. A. Minor. 2017. "Curvature-Based Ground Vehicle Control of Trailer Path Following Considering Sideslip and Limited Steering Actuation." *IEEE Transactions on Intelligent Transportation Systems* 18, no. 2: 332–348.
- Li, Z., M. V. Chitturi, A. R. Bill, and D. A. Noyce. 2012. "Automated Identification and Extraction of Horizontal Curve Information From Geographic Information System Roadway Maps." *Transportation Research Record* 2291: 80–92.
- Mathew, N., S. L. Smith, and S. L. Waslander. 2015. "Planning Paths for Package Delivery in Heterogeneous Multirobot Teams." *IEEE Transactions on Automation Science and Engineering* 12, no. 4: 1298–1308.
- Nourizadeh, P., F. J. Stevens McFadden, and W. N. Browne. 2023. "In Situ Slip Estimation for Mobile Robots in Outdoor Environments." *Journal of Field Robotics* 40, no. 3: 467–482.
- Park, M., S. Lee, and W. Han. 2015. "Development of Steering Control System for Autonomous Vehicle Using Geometry-Based Path Tracking Algorithm." *ETRI Journal* 37, no. 3: 617–625.
- Rubio, F., F. Valero, and C. Llopis-Albert. 2019. "A Review of Mobile Robots: Concepts, Methods, Theoretical Framework, and Applications." *International Journal of Advanced Robotic Systems* 16, no. 2: 1729881419839596.
- Safwat, E., W. Zhang, M. Wu, Y. Lyu, and Q. Jia. 2018. "Robust Path Following Controller for Unmanned Aerial Vehicle Based on Carrot Chasing Guidance Law Using Dynamic Inversion." In *2018 18th International Conference on Control, Automation and Systems (ICCAS)*, 1444–1450. IEEE.
- Samuel, M., M. Hussein, and M. B. Mohamad. 2016. "A Review of Some Pure-Pursuit Based Path Tracking Techniques for Control of Autonomous Vehicle." *International Journal of Computer Applications* 135, no. 1: 35–38.
- Siciliano, B., and O. Khatib. 2008. *Springer Handbook of Robotics*. Springer.
- Siciliano, B., L. Sciacivco, L. Villani, and G. Oriolo. 2010. *Robotics: Modelling, Planning and Control*. Springer Publishing Company Incorporated.
- Siegwart, R., I. R. Nourbakhsh, and D. Scaramuzza. 2011. *Introduction to Autonomous Mobile Robots* (2nd ed.). The MIT Press.
- Simba, K. R., N. Uchiyama, and S. Sano. 2016. "Real-Time Smooth Trajectory Generation for Nonholonomic Mobile Robots Using Bézier Curves." *Robotics and Computer-Integrated Manufacturing* 41: 31–42.
- Sujit, P. B., S. Saripalli, and J. B. Sousa. 2014. "Unmanned Aerial Vehicle Path Following: A Survey and Analysis of Algorithms for Fixed-Wing Unmanned Aerial Vehicless." *IEEE Control Systems* 34: 42–59.
- Thakur, D., M. Likhachev, and J. Keller, et al. 2013. "Planning for Opportunistic Surveillance With Multiple Robots." In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 5750–5757. Institute of Electrical and Electronics Engineers Inc.
- Villagra, J., V. Milanés, J. Pérez, and J. Godoy. 2012. "Smooth Path and Speed Planning for an Automated Public Transport Vehicle." *Robotics and Autonomous Systems* 60, no. 2: 252–265.
- Yao, W. 2021. "Guiding Vector Fields for Robot Motion Control." PhD thesis, University of Groningen.
- Yao, W., and M. Cao. 2020. "Path Following Control in 3D Using a Vector Field." *Automatica* 117: 108957.
- Yao, W., H. G. de Marina, B. Lin, and M. Cao. 2021. "Singularity-Free Guiding Vector Field for Robot Navigation." *IEEE Transactions on Robotics* 37, no. 4: 1206–1221.
- Yao, W., Y. A. Kapitanyuk, and M. Cao. 2018. "Robotic Path Following in 3D Using a Guiding Vector Field." In *2018 IEEE Conference on Decision and Control (CDC)*, 4475–4480. Institute of Electrical and Electronics Engineers Inc.
- Yuan, S., Z. Liu, Y. Sun, Z. Wang, and L. Zheng. 2023. "An Event-Triggered Trajectory Planning and Tracking Scheme for Automatic Berthing of Unmanned Surface Vessel." *Ocean Engineering* 273: 113964.